REPRÉSENTATION EN MÉMOIRE & PARCOURS DES GRAPHES

UBTM (Module : Recherche opérationnelle)

2024-2025

Représentation des Graphes

Il existe plusieurs façons classiques de représenter un graphe $G = (S, A)$: par un ensemble de listes d'adjacences, par une matrice d'adjacences, ou par une matrice d'incidence.

La représentation par listes d'adjacences est souvent préférée, car elle fournit un moyen peu encombrant de représenter les graphes peu denses (ceux pour qui $|A|$ est très inférieur à $|S|^2$).

Toutefois, la représentation par matrice d'adjacences sera préférable si le graphe est dense ($|A|$ est proche de $|S|^2$) ou quand on veut savoir rapidement s'il existe un arc entre deux sommets donnés.

Représentation matrice d'incidence

Considérons un graphe non orienté (resp. orienté) d'ordre n possédant m arêtes (resp. arcs), dont on a numéroté les n sommets et les m arêtes (resp. arcs). On peut le représenter par sa matrice d'incidence $A_{n\times m} = (a_{ij})$, qui est une matrice comportant n lignes et m colonnes.

Dans le cas non orienté, le coefficient de cette matrice situé à l'intersection de la i-ème ligne et de la j-ème colonne vaut

$$a_{ij} = \begin{cases} 1 & \text{si le } i\text{-ème sommet est une extrémité de la } j\text{-ème arête} \\ 0 & \text{sinon} \end{cases}$$

Représentation matrice d'incidence

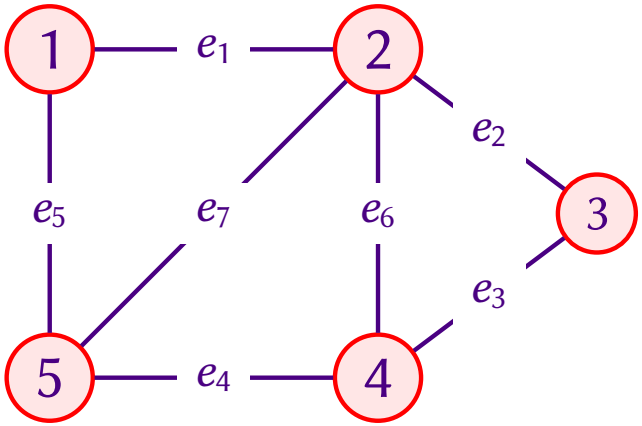
Dans le cas orienté, le coefficient de cette matrice situé à l'intersection de la i-ème ligne et de la j-ème colonne vaut

$$a_{ij} = \begin{cases} +1 & \text{si le } i\text{-ème sommet est l'origine du } j\text{-ème arc} \\ -1 & \text{si le } i\text{-ème sommet est l'extrémité du } j\text{-ème arc} \\ 0 & \text{sinon} \end{cases}$$

Dans une matrice d'incidence, le rôle des lignes et des colonnes n'est donc pas similaire, une ligne y représente un sommet et une colonne une arête ou arc.

Représentation matrice d'incidence

Graphe non orienté.

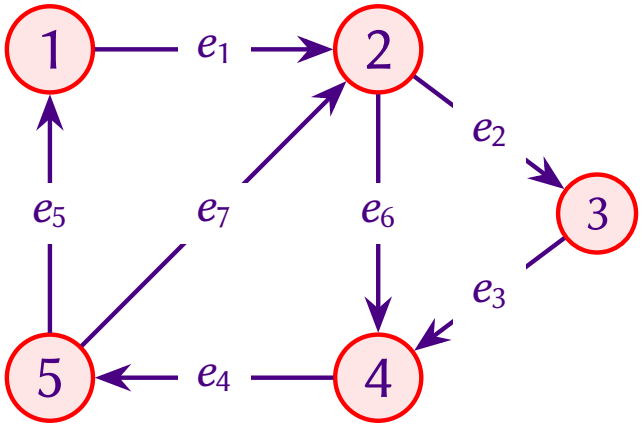


	e_1	e_2	e_3	e_4	e_5	e_6	e_7
1	1				1		
2	1	1				1	1
3		1	1				
4			1	1		1	
5				1	1		1

Espace utilisé $O(|S| \times |A|)$

Représentation matrice d'incidence

Graphe orienté.



	e_1	e_2	e_3	e_4	e_5	e_6	e_7
1	+1				-1		
2	-1	+1				+1	-1
3		-1	+1				
4			-1	+1		-1	
5				-1	+1		+1

Espace utilisé $O(|S| \times |A|)$

Représentation matrice d'incidence

Avantages et inconvénients

Cette matrice est essentiellement utilisée pour la modélisation de problèmes de programmation linéaire que l'on est amené à écrire pour résoudre des problèmes de graphes. C'est en particulier le cas des problèmes de flots dans les graphes.

Sous sa forme étendue, cette matrice est la plus gourmande en espace mémoire.

On n'a pas non plus directement l'information de l'existence ou non d'un arc entre i et j.

Représentation matrice d'adjacence

Aussi appelé matrice d'incidence sommets-sommets. Pour les graphes simples, l'ensemble des arcs définit une relation binaire entre les sommets que l'on peut représenter par sa matrice associée

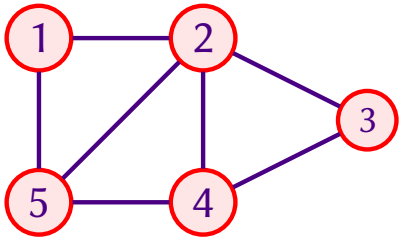
La matrice d'adjacence M d'un graphe $G = (S, A)$ d'ordre n est une matrice $n \times n$ a coefficients dans $\{0, 1\}$ ou chaque ligne et chaque colonne correspond a un sommet de G et ou :

$$M_{ij} = 1 \Longleftrightarrow (i, j) \in A$$

Dans le cas non orienté, la matrice d'adjacence est symétrique (une demi-matrice suffit).

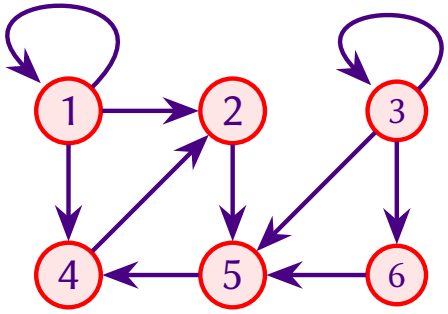
Représentation matrice d'adjacence

Graphe non orienté.



	1	2	3	4	5
1		1			1
2	1		1	1	1
3		1		1	
4		1	1		1
5	1	1		1	

Graphe orienté.



	1	2	3	4	5	6
1	1	1		1		
2					1	
3			1		1	1
4		1				
5				1		
6					1	

Espace utilisé $O(|S|^2)$

Représentation listes d'adjacence

Adaptée aux graphes peu denses. Étant donné un graphe $G = (S, A)$, on utilise :

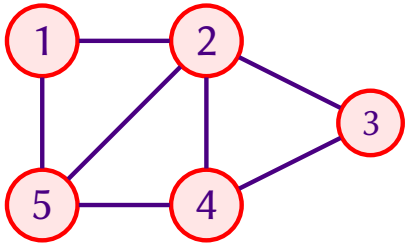
- un tableau Adj de $|S|$ listes, une pour chaque sommet.
- pour chaque sommet u , la liste $Adj[u]$ contient les sommets v adjacents à u (tels qu'il existe un arc/arête $(u, v) \in A$).

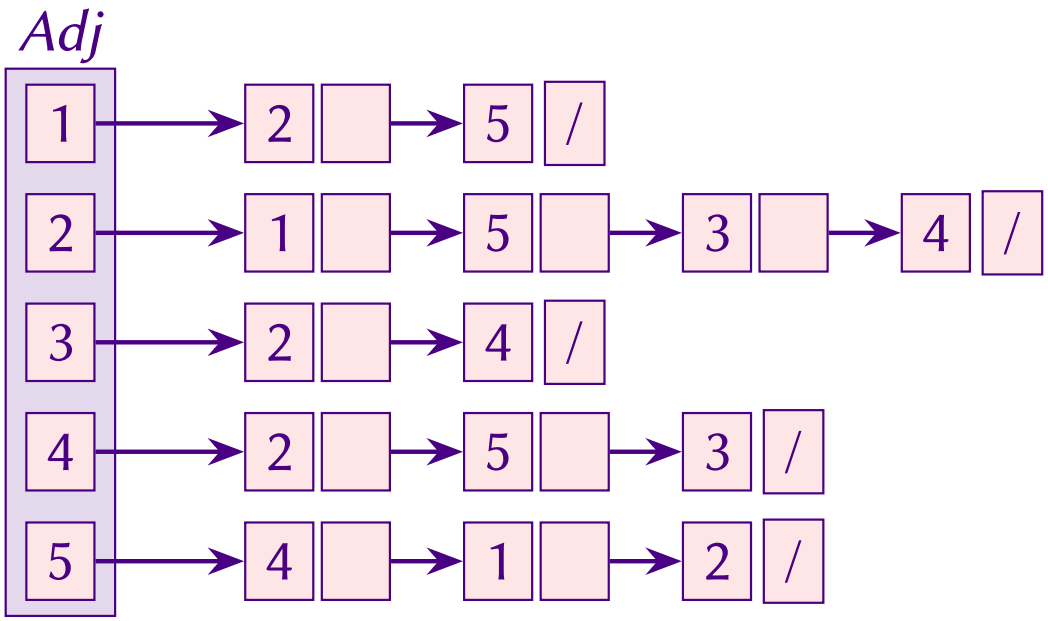
Pour un graphe oriente, la somme des longueurs des listes est $|A|$ chaque arc étant représente exactement une fois dans la liste du sommet origine.
Pour un graphe non oriente, elle est de $2|A|$ car chaque arête est représentée une fois dans chacun des 2 sommets de l'arête. Gros avantage, la taille mémoire de la structure est en $O(|S| + |A|)$.

inconvénient essentielle : lenteur de la résolution du problème de l'existence d'une arête entre deux sommets.

Représentation listes d'adjacence

Graphe non orienté.

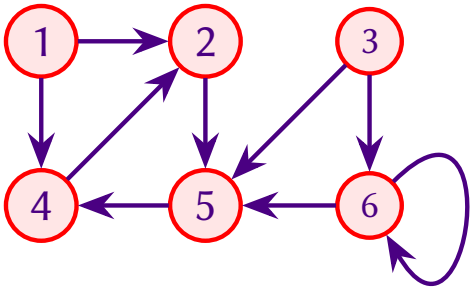


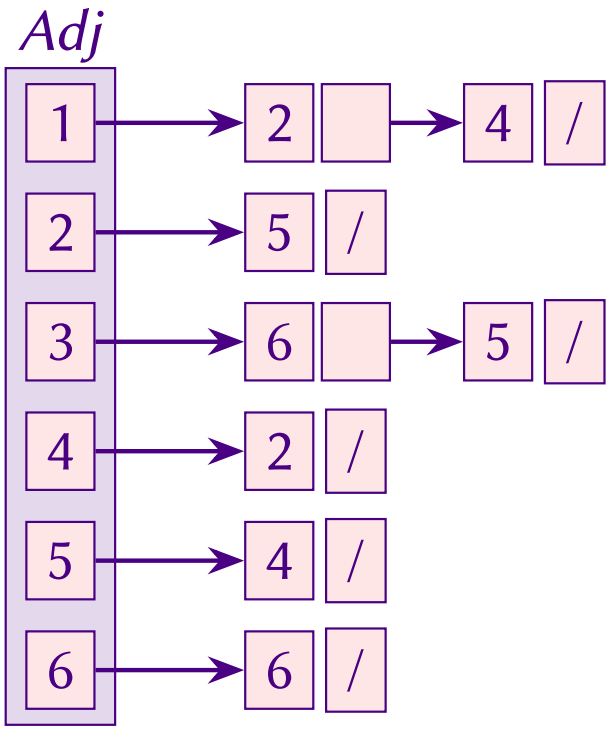


Espace utilisé $O(|S| + |A|)$.

Représentation listes d'adjacence

Graphe orienté.





Espace utilisé $O(|S| + |A|)$.

Parcours d'un graphe

Il s'agit d'écrire un algorithme qui permet d'examiner les sommets une et une seule fois. La présence de circuits doit être prise en considération de façon à ne pas visiter plusieurs fois le même sommet. Il faut donc marquer les sommets déjà visités.

On distingue deux types de parcours :

- le parcours en profondeur (DFS : depth first search)
- le parcours en largeur (BFS : breadth first search).

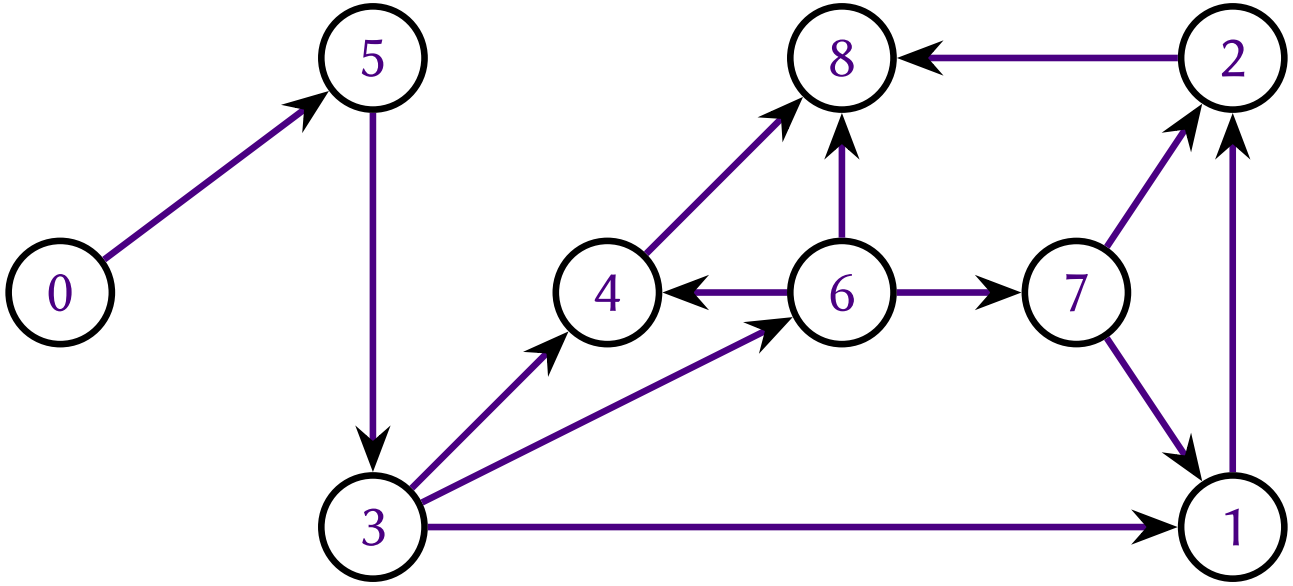
Parcours en profondeur d'abord (DFS : Depth First Search)

Principe du parcours en profondeur d'un graphe :

La stratégie suivie par un parcours en profondeur est, comme son nom l'indique, de descendre plus « profondément » dans le graphe chaque fois que c'est possible.

On part d'un sommet donné. On énumère le premier fils de ce sommet (par ordre alphabétique par exemple), puis on repart de ce dernier sommet pour atteindre le premier petit-fils, etc. Il s'agit pour chaque sommet visité, de choisir un des sommets successeurs du sommet en cours, jusqu'à arriver sur une impasse ou un sommet déjà visité. Dans ce cas, on revient en arrière pour repartir avec un des successeurs non visité du sommet courant.

Parcours en profondeur d'abord (DFS)

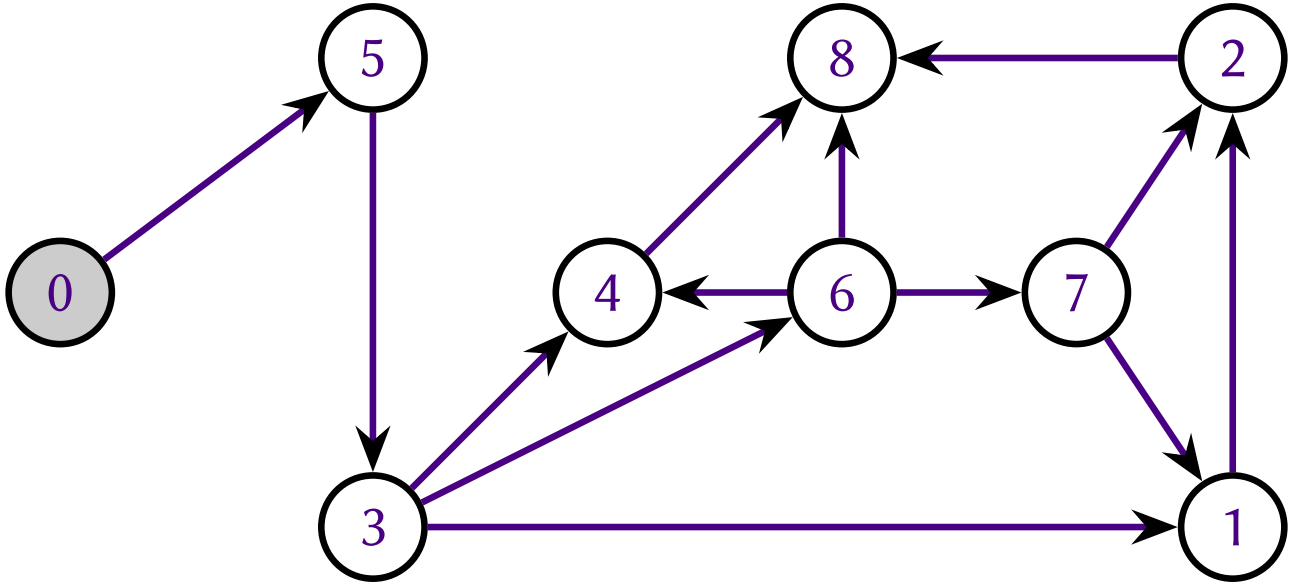


	0	1	2	3	4	5	6	7	8
Couleur	B	B	B	B	B	B	B	B	B

Pile	

Output :

Parcours en profondeur d'abord (DFS)

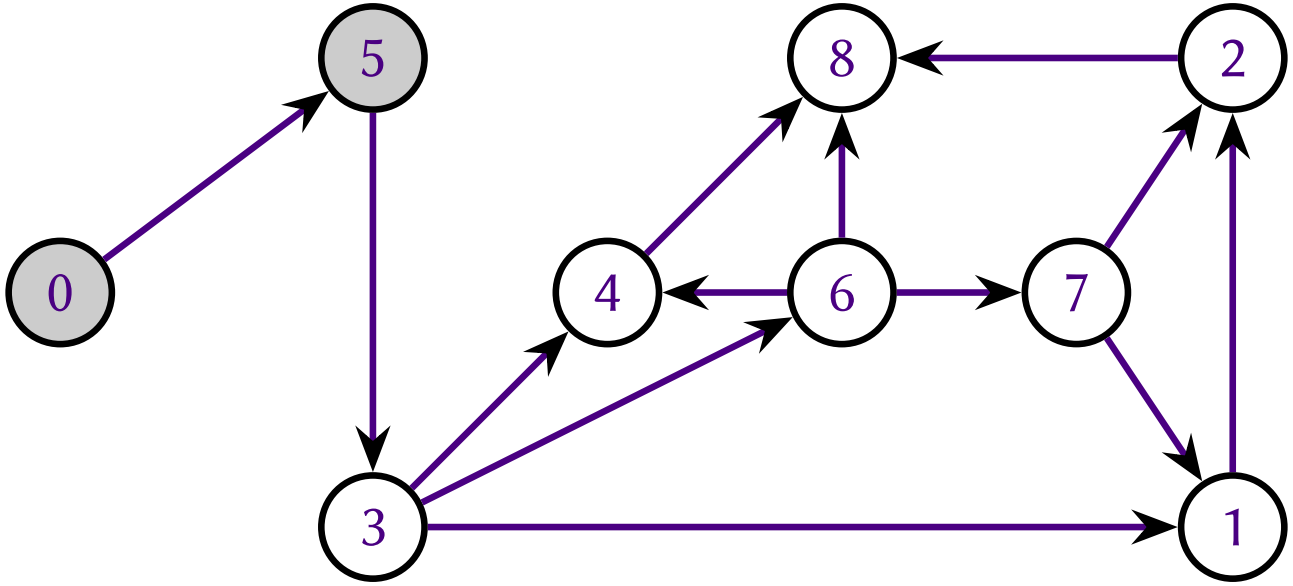


	0	1	2	3	4	5	6	7	8
Couleur	G	B	B	B	B	B	B	B	B

Pile	0

Output : 0

Parcours en profondeur d'abord (DFS)

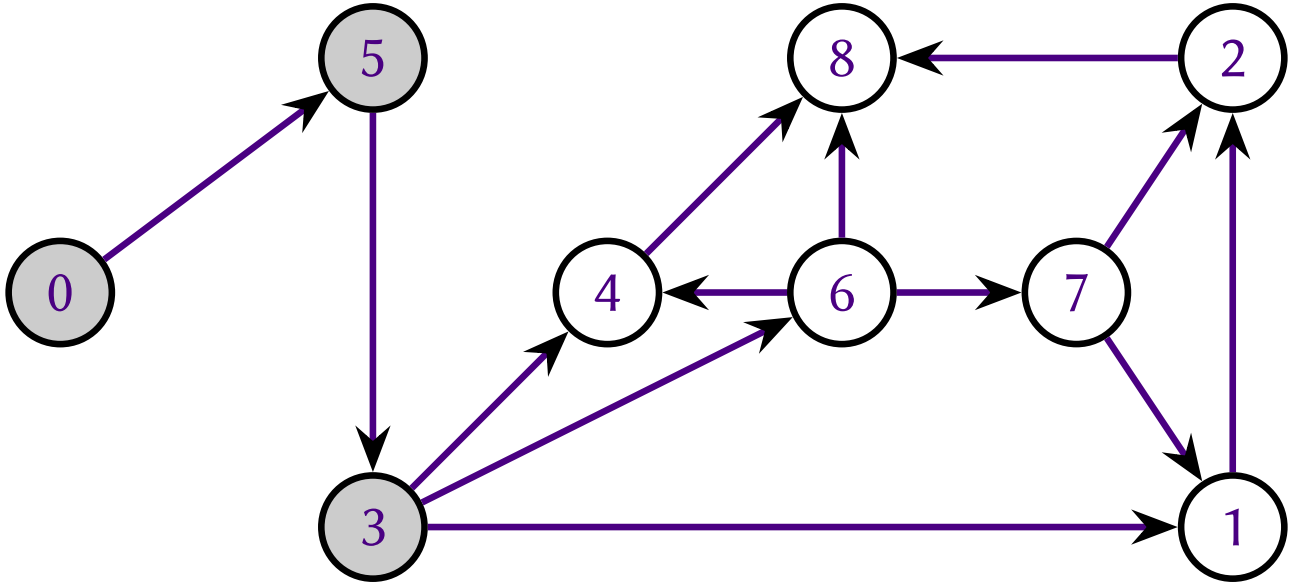


	0	1	2	3	4	5	6	7	8
Couleur	G	B	B	B	B	G	B	B	B

Pile	0	5

Output : 0 5

Parcours en profondeur d'abord (DFS)

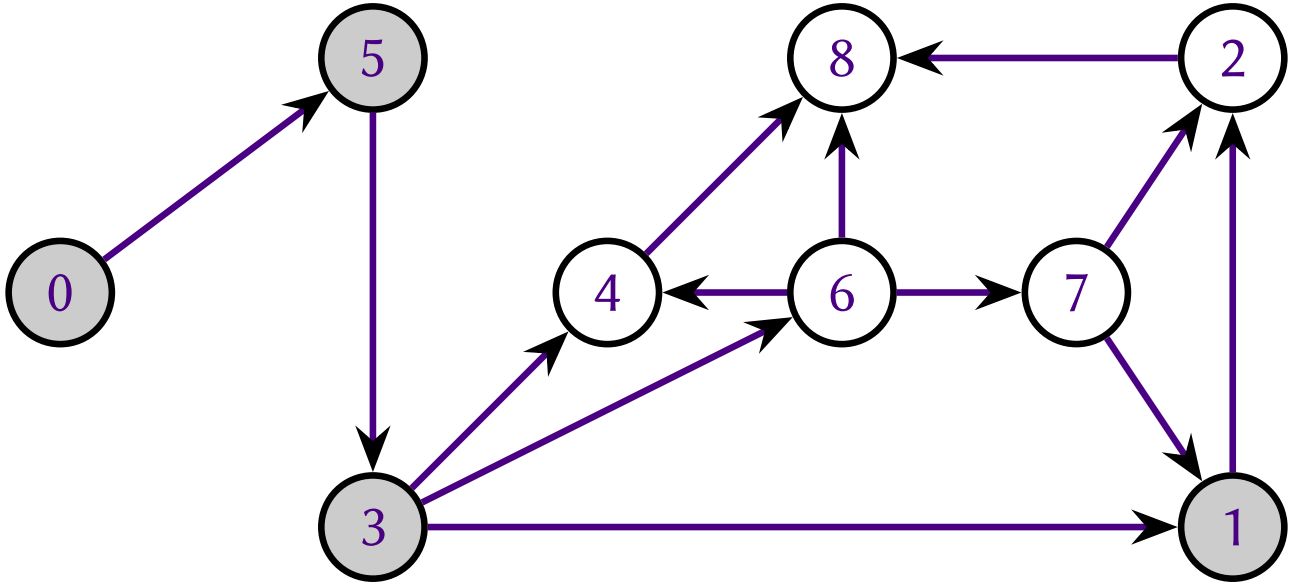


	0	1	2	3	4	5	6	7	8
Couleur	G	B	B	G	B	G	B	B	B

Pile	0	5	3

Output : 0 5 3

Parcours en profondeur d'abord (DFS)

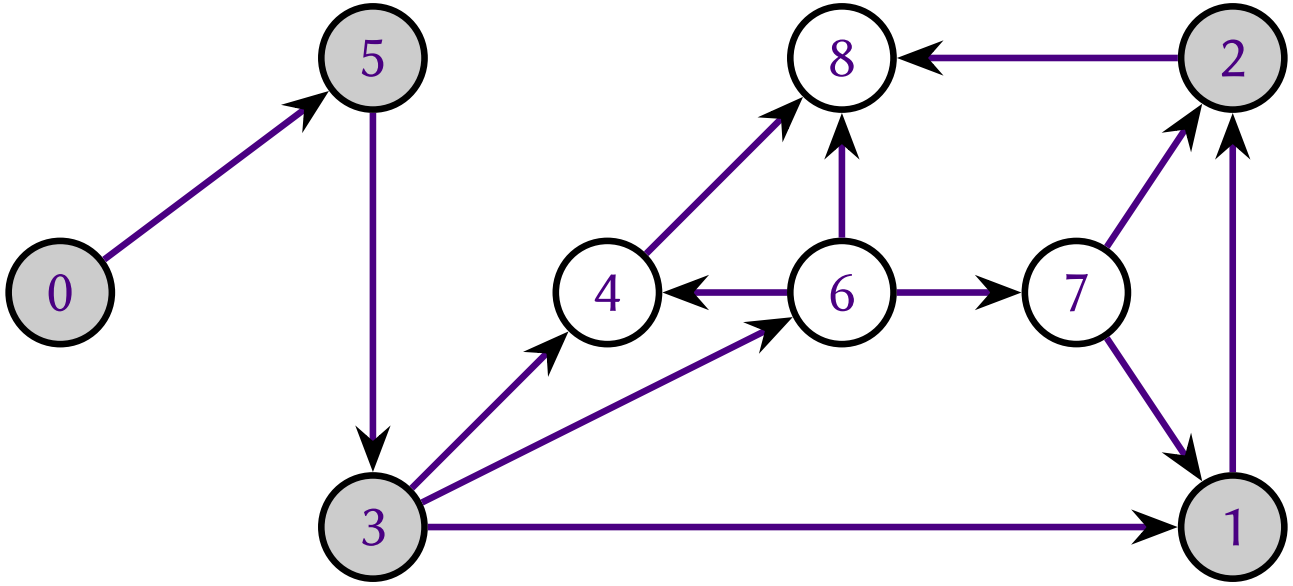


	0	1	2	3	4	5	6	7	8
Couleur	G	G	B	G	B	G	B	B	B

Pile	0	5	3	1

Output : 0 5 3 1

Parcours en profondeur d'abord (DFS)

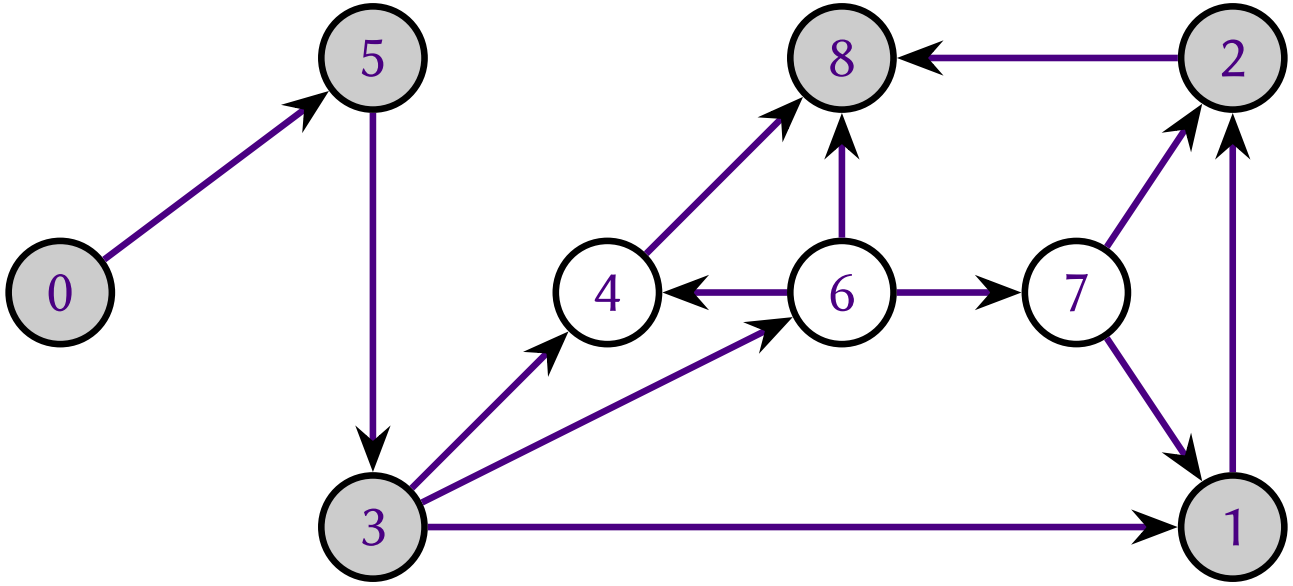


	0	1	2	3	4	5	6	7	8
Couleur	G	G	G	G	B	G	B	B	B

Pile	0	5	3	1	2

Output : 0 5 3 1 2

Parcours en profondeur d'abord (DFS)

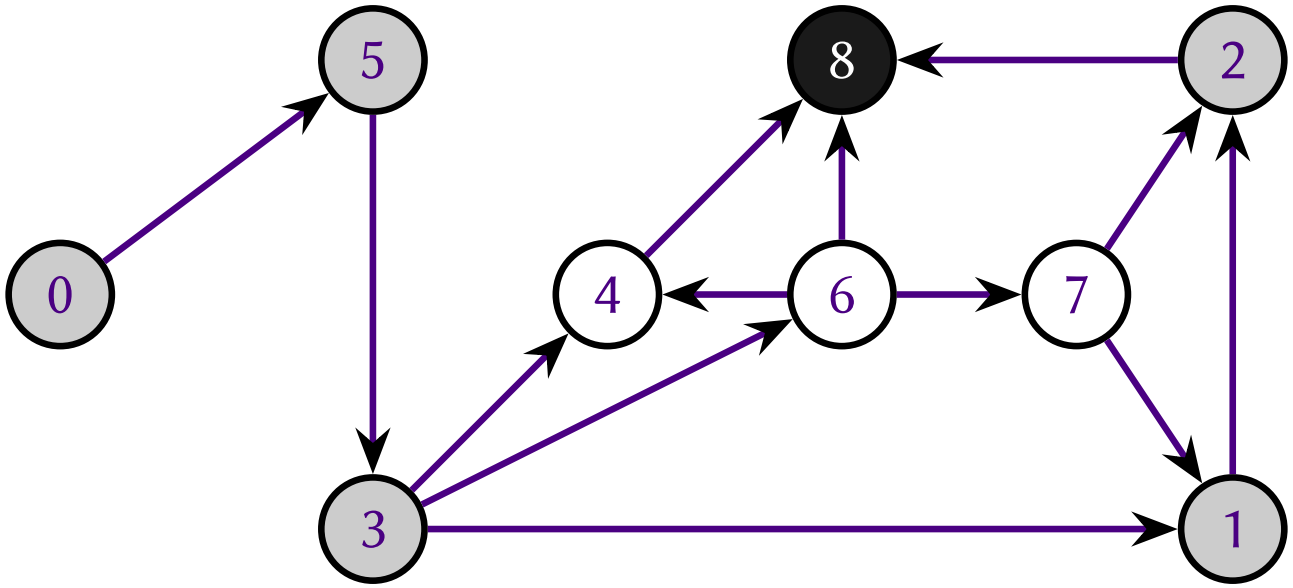


	0	1	2	3	4	5	6	7	8
Couleur	G	G	G	G	B	G	B	B	G

Pile	0	5	3	1	2	8

Output : 0 5 3 1 2 8

Parcours en profondeur d'abord (DFS)

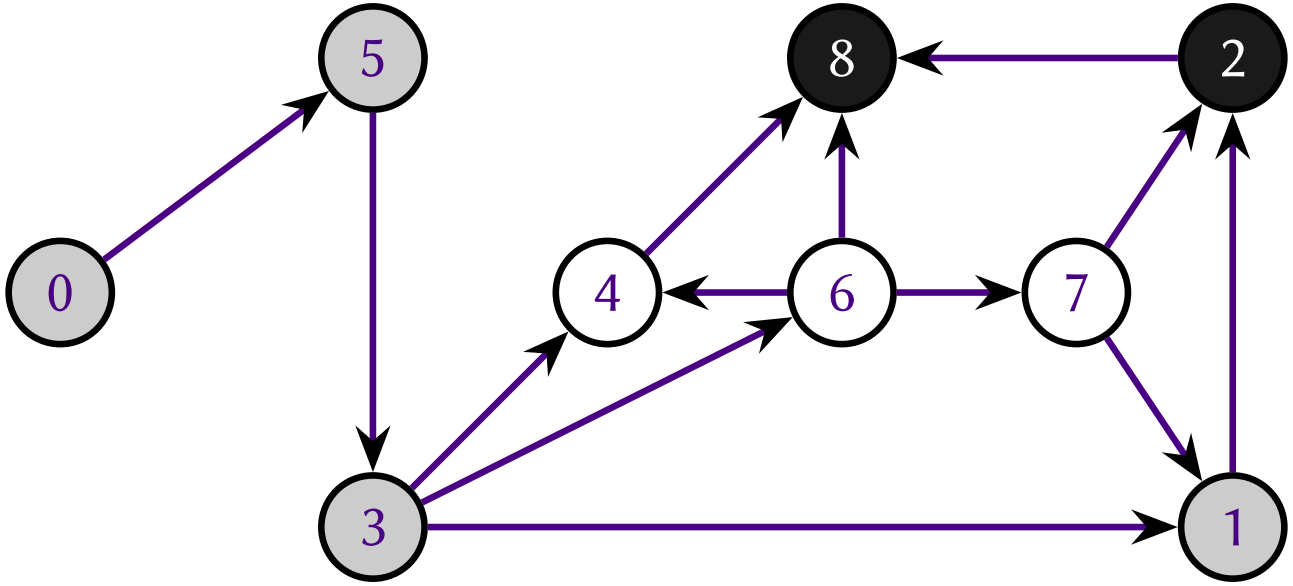


	0	1	2	3	4	5	6	7	8
Couleur	G	G	G	G	B	G	B	B	N

Pile	0	5	3	1	2

Output : 0 5 3 1 2 8

Parcours en profondeur d'abord (DFS)

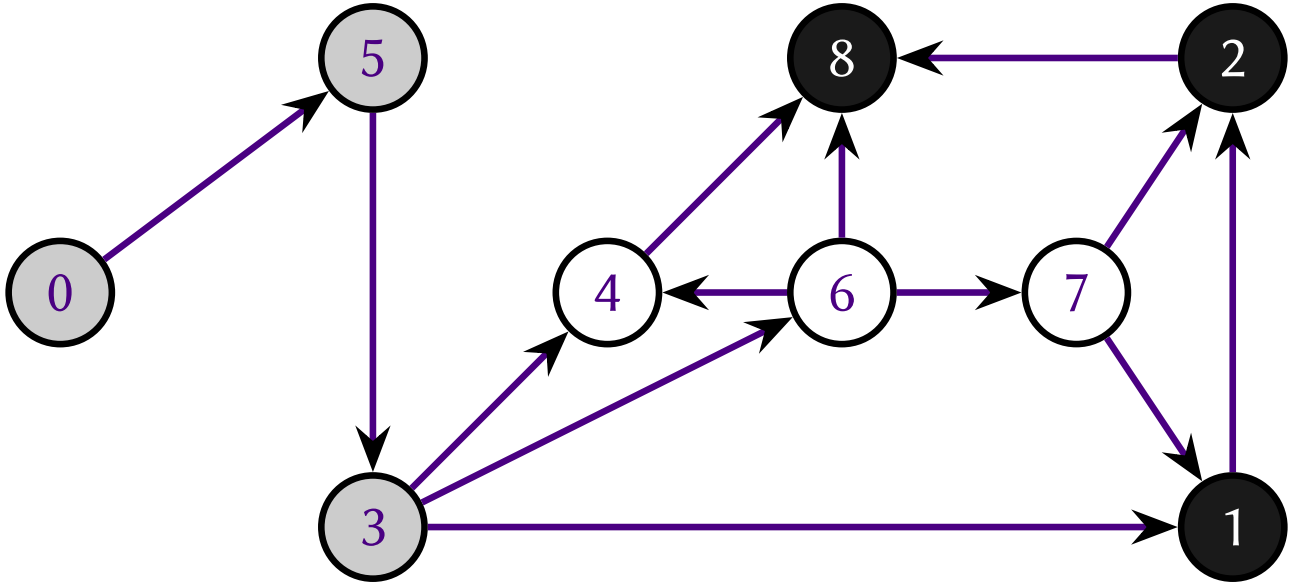


	0	1	2	3	4	5	6	7	8
Couleur	G	G	N	G	B	G	B	B	N

Pile	0	5	3	1

Output : 0 5 3 1 2 8

Parcours en profondeur d'abord (DFS)

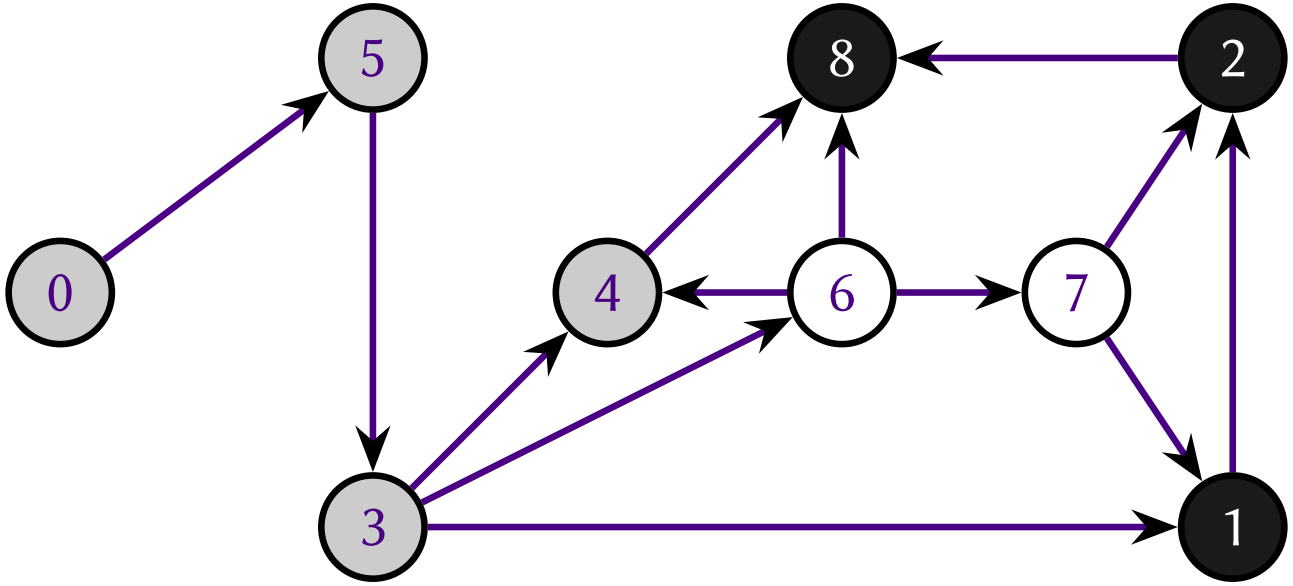


	0	1	2	3	4	5	6	7	8
Couleur	G	N	N	G	B	G	B	B	N

Pile	0	5	3

Output : 0 5 3 1 2 8

Parcours en profondeur d'abord (DFS)

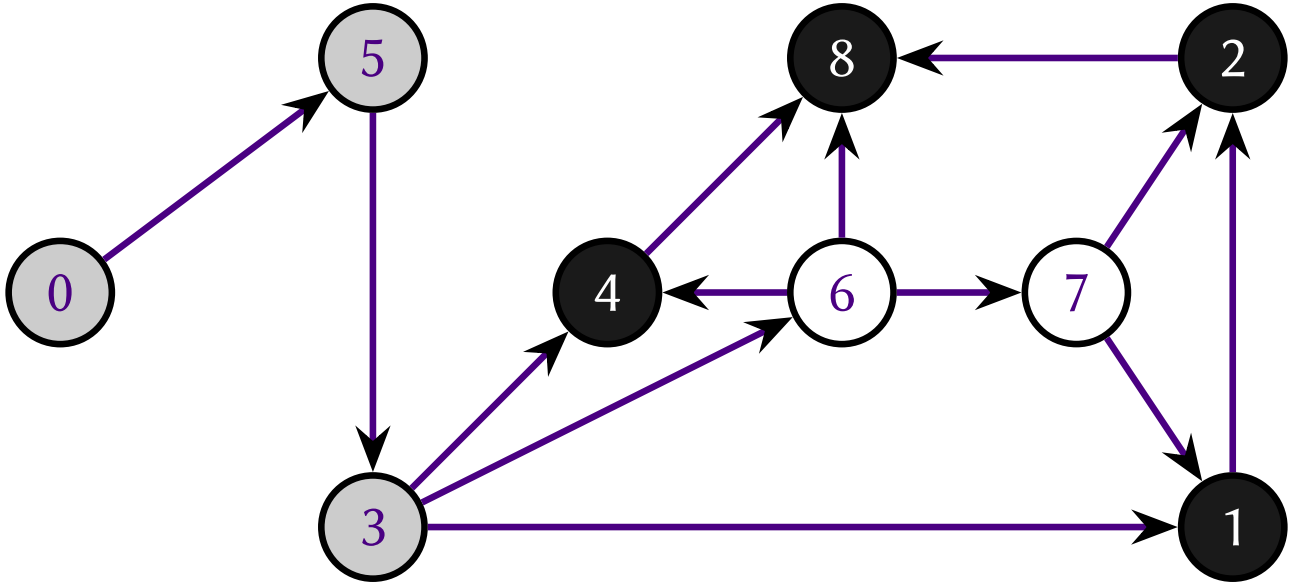


	0	1	2	3	4	5	6	7	8
Couleur	G	N	N	G	G	G	B	B	N

Pile	0	5	3	4					

Output : 0 5 3 1 2 8 4

Parcours en profondeur d'abord (DFS)

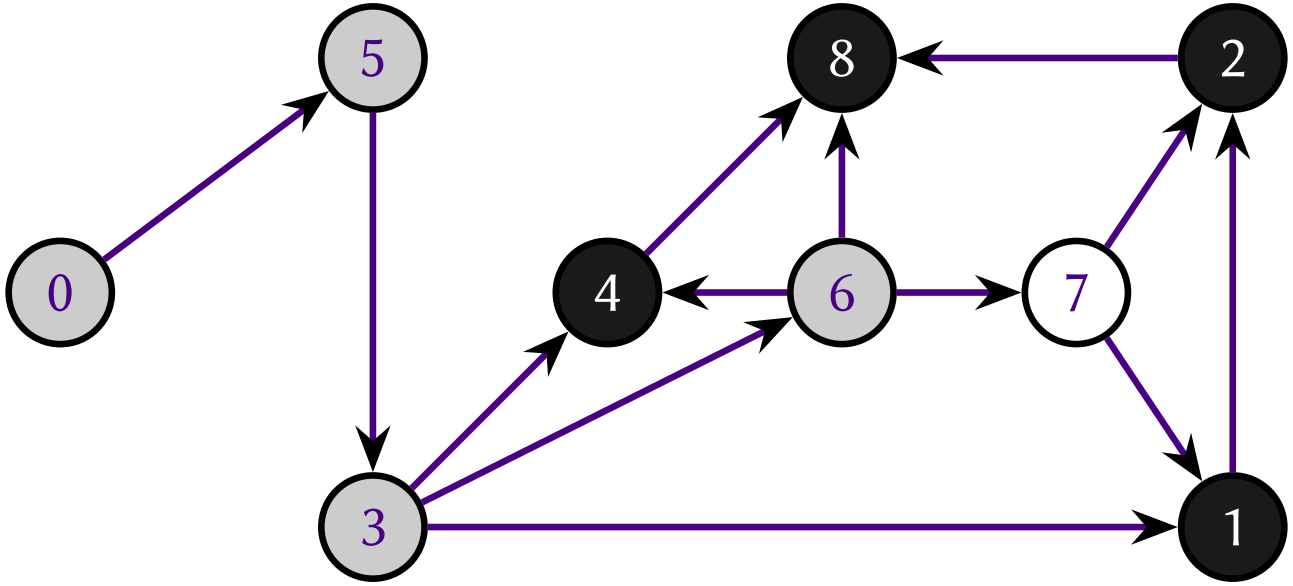


	0	1	2	3	4	5	6	7	8
Couleur	G	N	N	G	N	G	B	B	N

Pile	0	5	3

Output : 0 5 3 1 2 8 4

Parcours en profondeur d'abord (DFS)

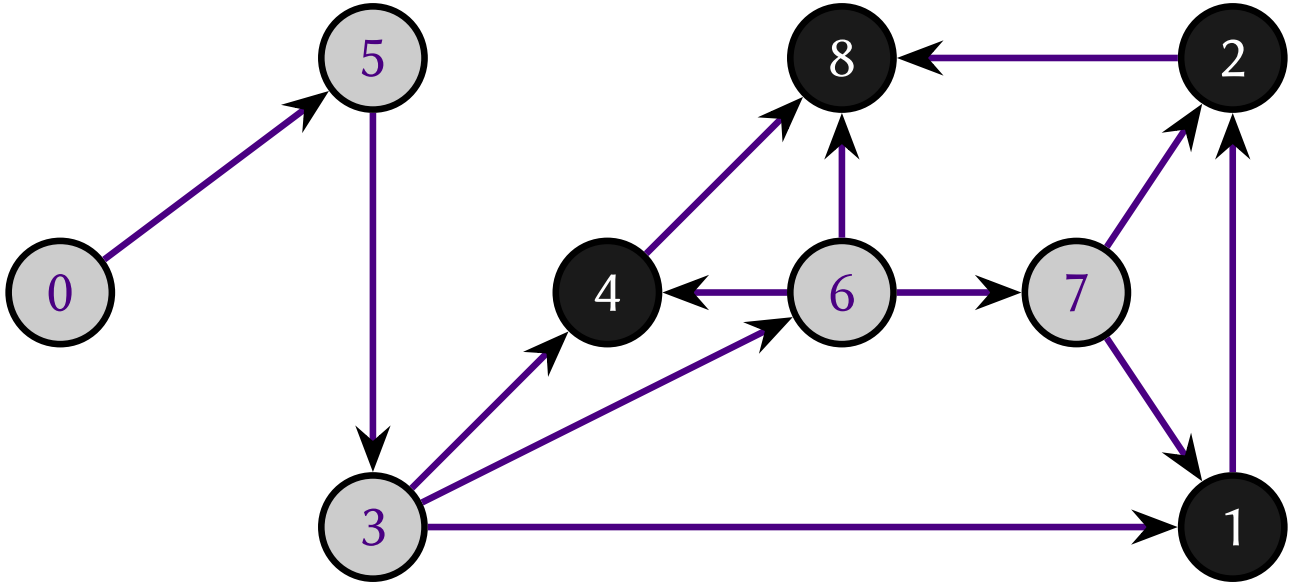


	0	1	2	3	4	5	6	7	8
Couleur	G	N	N	G	N	G	G	B	N

Pile	0	5	3	6

Output : 0 5 3 1 2 8 4 6

Parcours en profondeur d'abord (DFS)

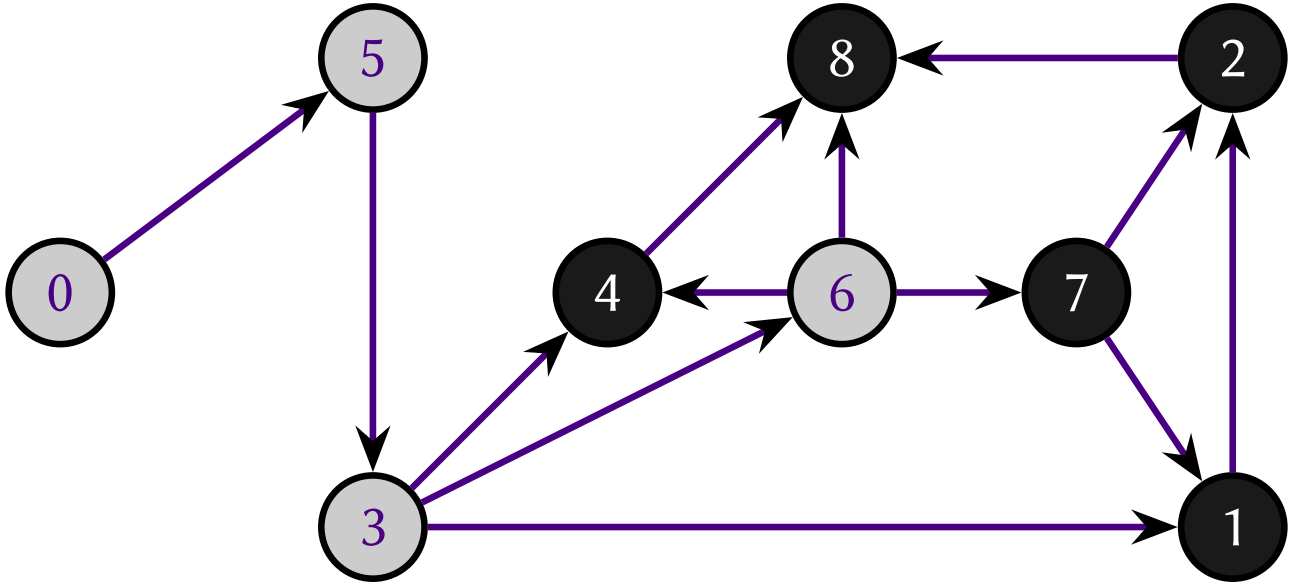


	0	1	2	3	4	5	6	7	8
Couleur	G	N	N	G	N	G	G	G	N

Pile	0	5	3	6	7

Output : 0 5 3 1 2 8 4 6 7

Parcours en profondeur d'abord (DFS)

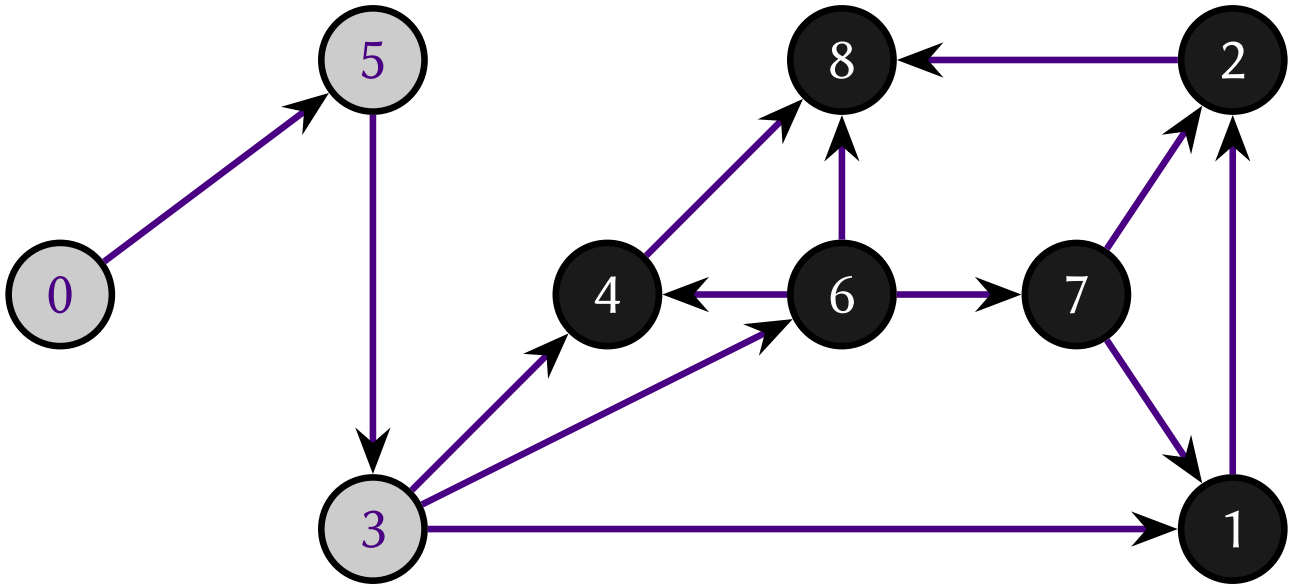


	0	1	2	3	4	5	6	7	8
Couleur	G	N	N	G	N	G	G	N	N

Pile	0	5	3	6

Output : 0 5 3 1 2 8 4 6 7

Parcours en profondeur d'abord (DFS)

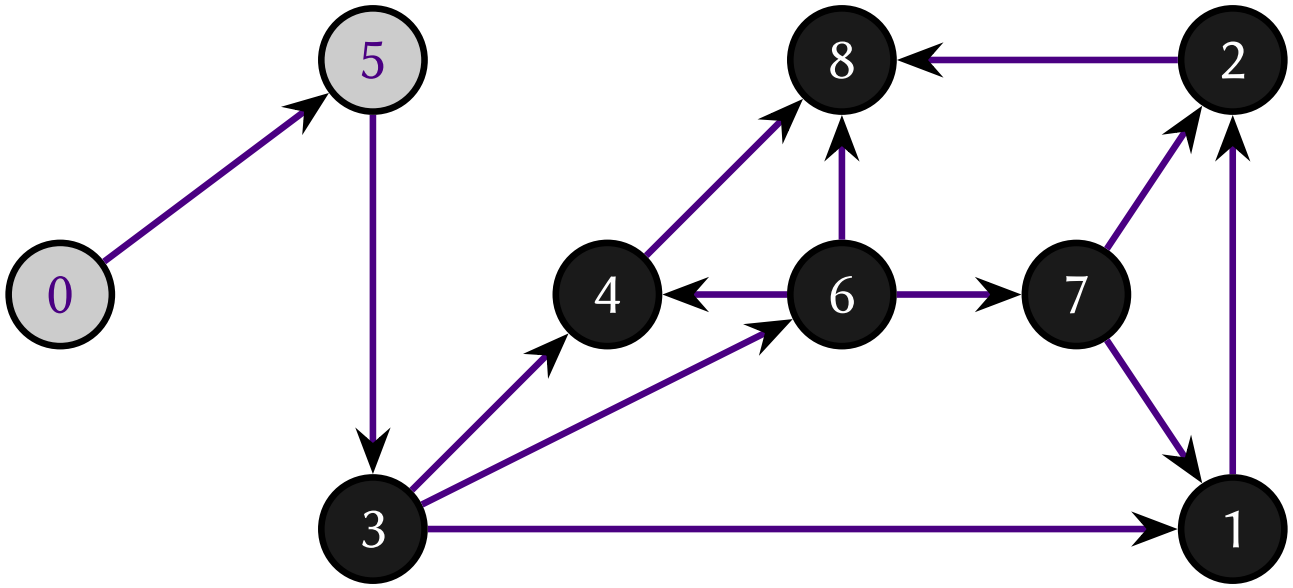


	0	1	2	3	4	5	6	7	8
Couleur	G	N	N	G	N	G	N	N	N

Pile	0	5	3

Output : 0 5 3 1 2 8 4 6 7

Parcours en profondeur d'abord (DFS)

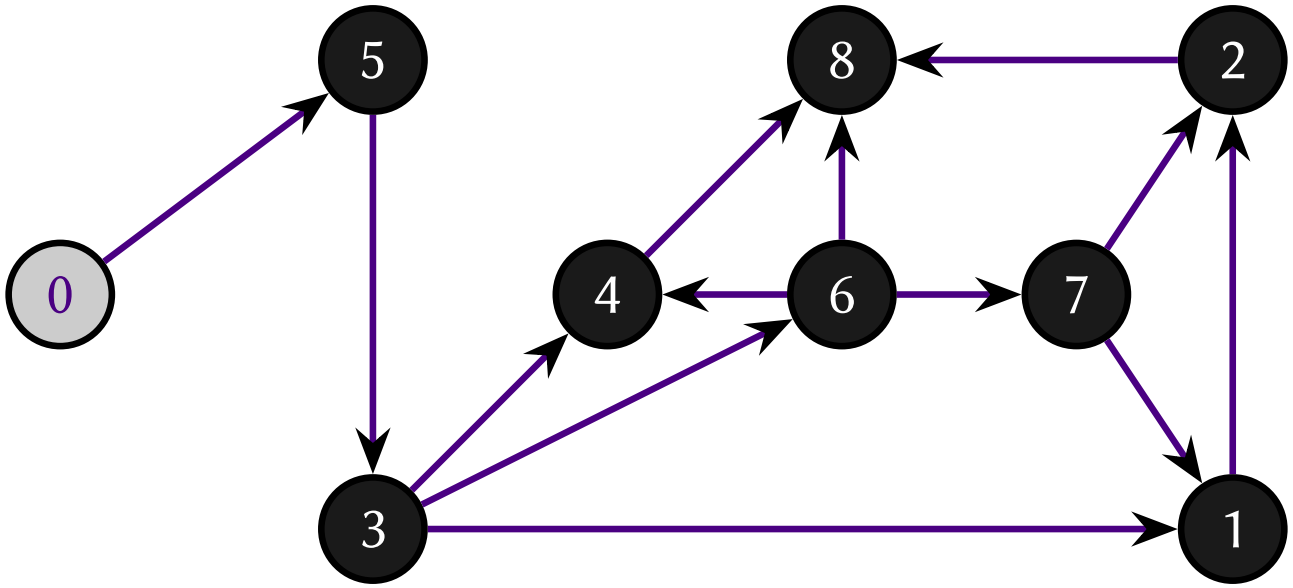


	0	1	2	3	4	5	6	7	8
Couleur	G	N	N	N	N	G	N	N	N

Pile	0	5

Output : 0 5 3 1 2 8 4 6 7

Parcours en profondeur d'abord (DFS)

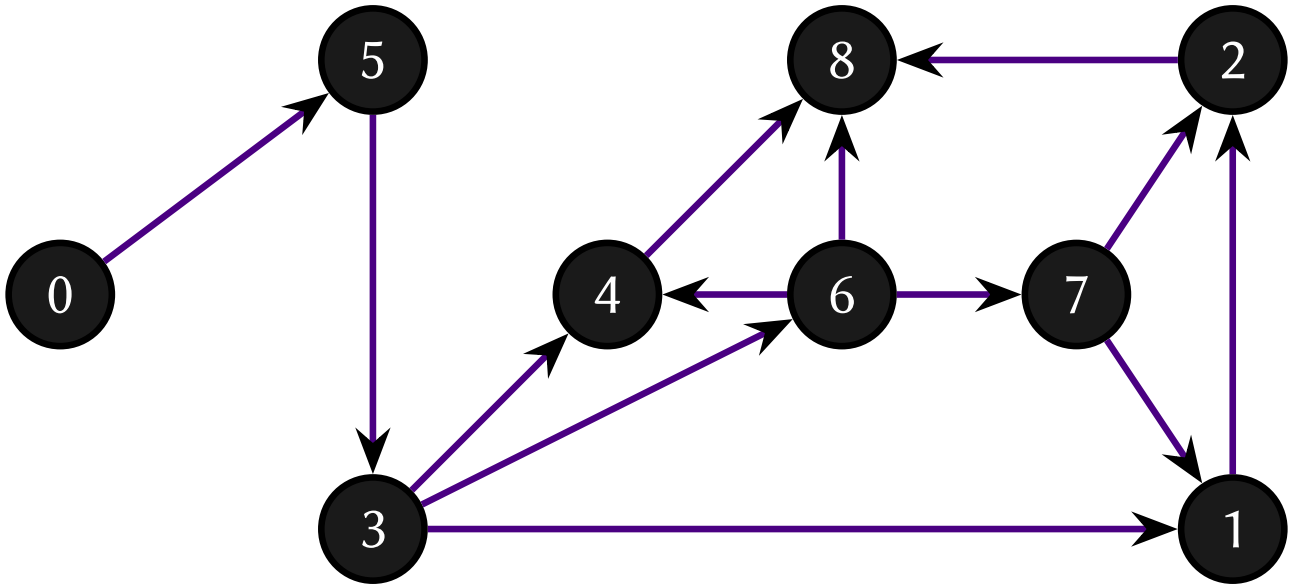


	0	1	2	3	4	5	6	7	8
Couleur	G	N	N	N	N	N	N	N	N

Pile	0

Output : 0 5 3 1 2 8 4 6 7

Parcours en profondeur d'abord (DFS)

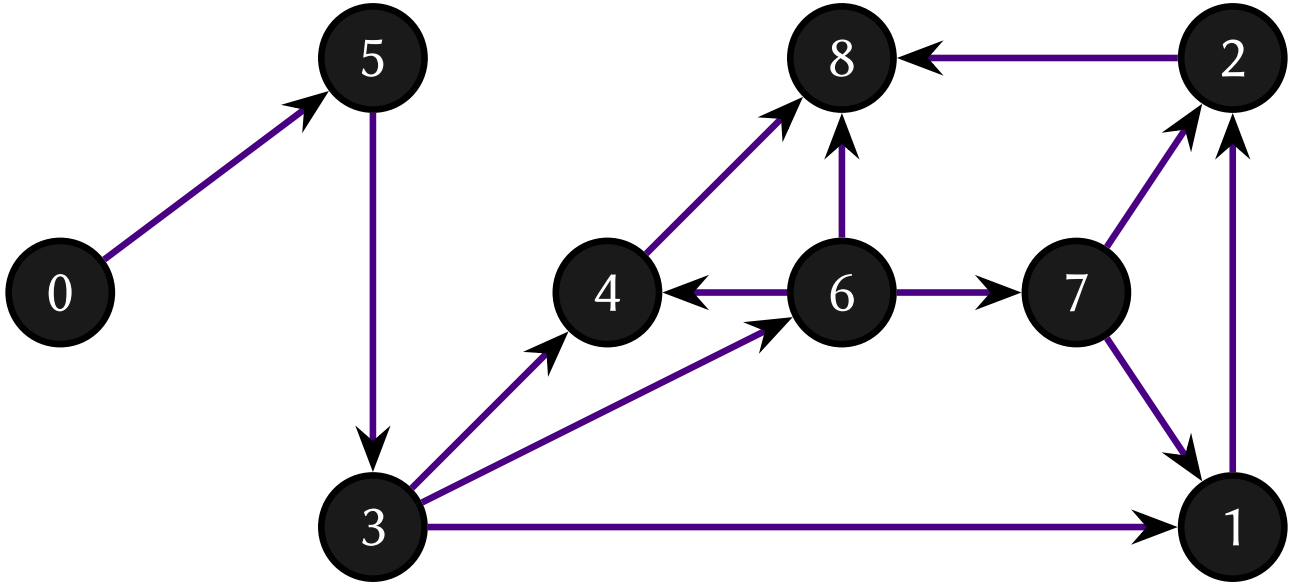


	0	1	2	3	4	5	6	7	8
Couleur	N	N	N	N	N	N	N	N	N

Pile	

Output : 0 5 3 1 2 8 4 6 7

Parcours en profondeur d'abord (DFS)



	0	1	2	3	4	5	6	7	8
Couleur	N	N	N	N	N	N	N	N	N

Pile	

Output : 8 2 1 4 7 6 3 5 0

Parcours en profondeur d'abord (DFS)

Algorithme 1 Parcours en profondeur d'un Graphe G

```
Procedure DFS(G)
    Pour chaque sommet u ∈ S faire {
        couleur[u] ← b ;
        π[u] ← NIL ; }
    t ← 0 ;
    Pour chaque sommet u ∈ S faire
        si couleur[u] = b alors DFS_Visit(u)

    Procedure DFS_Visit(u)
    couleur[u] ← g ;
    d[u] ← (t ← (t + 1)) ;
    Pour chaque sommet v ∈ Adj[u] faire
        si couleur[v] = b alors {
            π[v] ← u ;
            DFS_Visit(v) ; }
    couleur[u] ← n ;
    f[u] ← (t ← (t + 1)) ;
```

Parcours en profondeur d'abord (DFS)

Structures de données utilisées :

- On utilise, un tableau π qui associe à chaque sommet le sommet qui l'a fait entrer dans la pile, et un tableau *couleur* qui associe à chaque sommet sa couleur (blanc, gris ou noir).
- On va en plus mémoriser pour chaque sommet u :
  - $d[u]$ = date de découverte de u (passage en gris)
  - $f[u]$ = date de fin de traitement de u (passage en noir)
- où l'unité de temps est une itération. La date courante est mémorisée dans la variable t.

Définition (numérotation préfixe et suffixe) : On appelle numérotation préfixe et numérotation suffixe les numérotations des sommets du graphe correspondants à l'ordre de traitement des sommets du graphe lors du parcours en profondeur suivant qu'on numérote un sommet avant (gris) ou après le traitement de ses successeurs (noirs) :

Quel est le temps de calcul de cet algorithme ?

L'initialisation dans DFS parcourt chaque sommet donc en $O(|S|)$. La procédure DFS-Visit est appelée une fois pour chaque sommet puisqu'elle n'est invoquée que sur les sommets blancs et que la première chose qu'elle fait est de les colorier en gris.

Durant l'exécution de DFS-Visit une boucle s'effectue sur les sommets adjacents. Comme la somme des degrés (sortants) de tous les sommets est en $O(|A|)$, le coût total de DFS-Visit est en $O(|A|)$.

Globalement, DFS est donc en $O(|S| + |A|)$ et est donc linéaire dans la taille du graphe (sous forme de listes d'adjacence).

Parcours en profondeur d'abord (DFS)

```
#include <iostream>
#include <vector>
#include <list>
#include <iomanip>

#define Nil -1
#define Blanc 'B'
#define Gris 'G'
#define Noir 'N'

using namespace std;

vector<char> couleur;
vector<int> d;
vector<int> f;
vector<int> Pi;
int date;

void DFS(vector<list<int> > Graphe_G)
{
    void DFS_Visit(vector<list<int> > Graphe_G, int u);
    for (unsigned u = 0; u < Graphe_G.size(); u++) {
        couleur.push_back(Blanc);
        d.push_back(0);
        f.push_back(0);
        Pi.push_back(Nil);
    }
    date = 0;
    for (unsigned u = 0; u < Graphe_G.size(); u++)
        if (couleur[u] == Blanc) {
```

Parcours en profondeur d'abord (DFS)

```
            cout << "\t\t";
            DFS_Visit(Graphe_G, u);
            cout << endl;
        }
}

void DFS_Visit(vector<list<int> > Graphe_G, int u)
{
    int v;
    couleur[u] = Gris;
    cout << u << "  "; // traitement en numérotation préfixe
    d[u] = date = date + 1;
    list<int>::iterator index;
    for (index = Graphe_G[u].begin(); index != Graphe_G[u].end(); ++index) {
        v = *index;
        if (couleur[v] == Blanc) {
            Pi[v] = u;
            DFS_Visit(Graphe_G, v);
        }
    }
    couleur[u] = Noir;
    //cout << u << "  ";  // traitement en numérotation suffixe
    f[u] = date = date + 1;
}
void initialisation_graphe(vector<list<int> >& Graphe)
{
    Graphe[0].push_back(5);
    Graphe[5].push_back(3);
    Graphe[3].push_back(1);
    Graphe[3].push_back(4);
```

Parcours en profondeur d'abord (DFS)

```
    Graphe[3].push_back(6);
    Graphe[1].push_back(2);
    Graphe[4].push_back(8);
    Graphe[6].push_back(4);
    Graphe[6].push_back(7);
    Graphe[6].push_back(8);
    Graphe[7].push_back(1);
    Graphe[7].push_back(2);
    Graphe[2].push_back(8);
    Graphe[9].push_back(10);
}

int main()
{
    unsigned N = 11; // le nombre de Sommets du graphe
    vector<list<int> > Graphe(N); // déclaration du graphe en list d'adjacences

    initialisation_graphe(Graphe);

    //*********************************************************************
    int compteur1 = 0;
    cout << "\n\n  Le Graphe représenter par les listes d'adjacences :\n\n";
    for (unsigned i = 0; i < Graphe.size(); ++i) {
        list<int>::iterator iter;
        cout << "\tGraphe [" << i << "] \t: ";
        for (iter = Graphe[i].begin(); iter != Graphe[i].end(); ++iter) {
            cout << "(" << *iter << ")"
                 << "->";
            compteur1++;
        }
```

Parcours en profondeur d'abord (DFS)

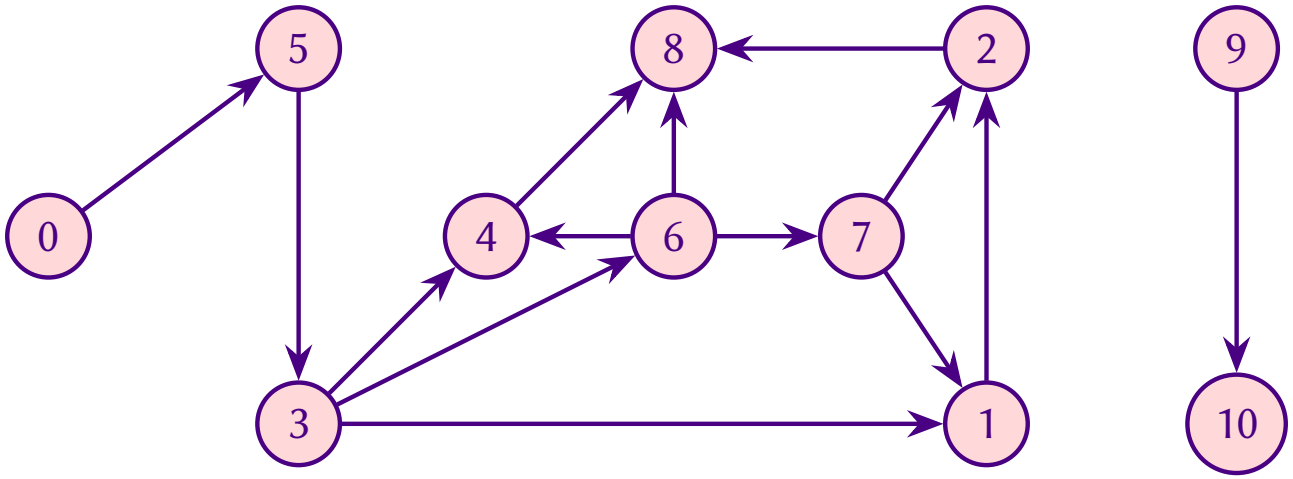
```
    cout << "(/)" << endl;
}

cout << "\n  Nombre d'arcs     : " << compteur1 << "\n";
cout << "  Nombre de sommets : " << N << "\n\n";

cout << "  Parcours en Profondeur du graphe avec numérotation préfixe :";
cout << endl
     << endl;

//*********************************************************************

DFS(Graphe); // appelle de la precedure DFS

//*********************************************************************

cout << "\n  Les Sommets     :\t"
     << "| ";
for (unsigned j = 0; j < Graphe.size(); j++)
    cout << setw(2) << j << " | ";
cout << "\n  Date découverte :\t"
     << "| ";
for (unsigned j = 0; j < Graphe.size(); j++)
    cout << setw(2) << d[j] << " | ";
cout << endl;
cout << "  Date de fin     :\t"
     << "| ";
for (unsigned j = 0; j < Graphe.size(); j++)
    cout << setw(2) << f[j] << " | ";
cout << endl;
```

Parcours en profondeur d'abord (DFS)

```
    cout << "  Arborescence    :\t"
         << "| ";
    for (unsigned j = 0; j < Graphe.size(); j++)
        cout << setw(2) << Pi[j] << " | ";
    cout << endl;
    cout << "  Couleur         :\t"
         << "| ";
    for (unsigned j = 0; j < Graphe.size(); j++)
        cout << setw(2) << couleur[j] << " | ";
    cout << endl
         << endl;
    return 0;
}
```

Parcours en profondeur d'abord (DFS)

En appliquant le parcours en profondeur DFS sur le graphe suivant :



Parcours en profondeur d'abord (DFS)

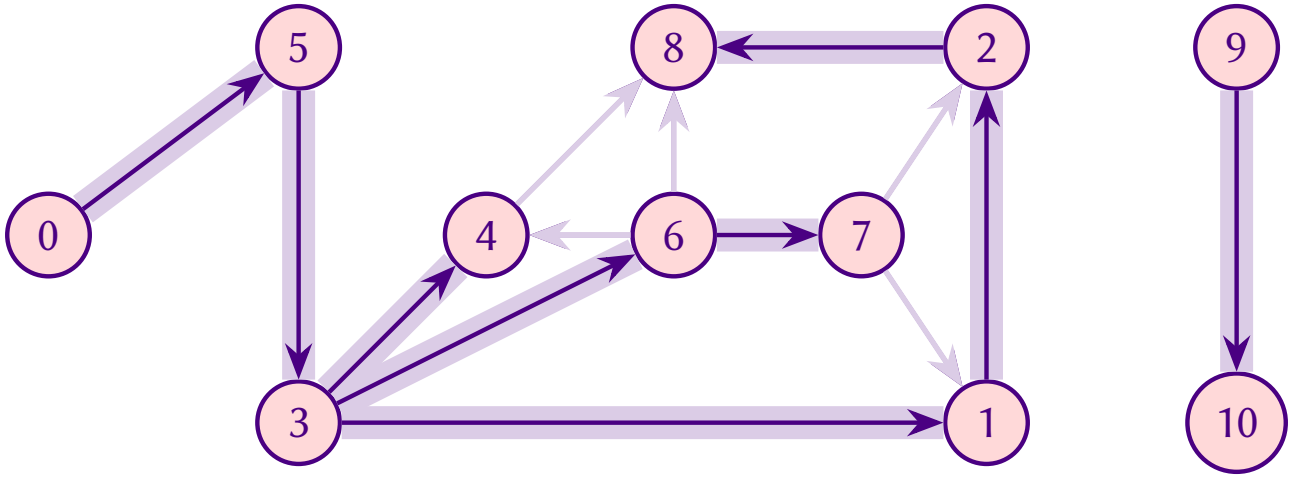
Le Graphe représenter par les listes d'adjacences :

```
  Graphe [0]   : (5)->(/)
  Graphe [1]   : (2)->(/)
  Graphe [2]   : (8)->(/)
  Graphe [3]   : (1)->(4)->(6)->(/)
  Graphe [4]   : (8)->(/)
  Graphe [5]   : (3)->(/)
  Graphe [6]   : (4)->(7)->(8)->(/)
  Graphe [7]   : (1)->(2)->(/)
  Graphe [8]   : (/)
  Graphe [9]   : (10)->(/)
  Graphe [10]  : (/)

Nombre d'arcs     : 14
Nombre de sommets : 11

Parcours en Profondeur du graphe avec numérotation préfixe :

      0  5  3  1  2  8  4  6  7
      9  10
```

Les Sommets	0	1	2	3	4	5	6	7	8	9	10
Date découverte	1	4	5	3	10	2	12	13	6	19	20
Date de fin	18	9	8	16	11	17	15	14	7	22	21
Arborescence	-1	3	1	5	3	0	3	6	2	-1	9
Couleur	N	N	N	N	N	N	N	N	N	N	N

Parcours en profondeur d'abord (DFS)

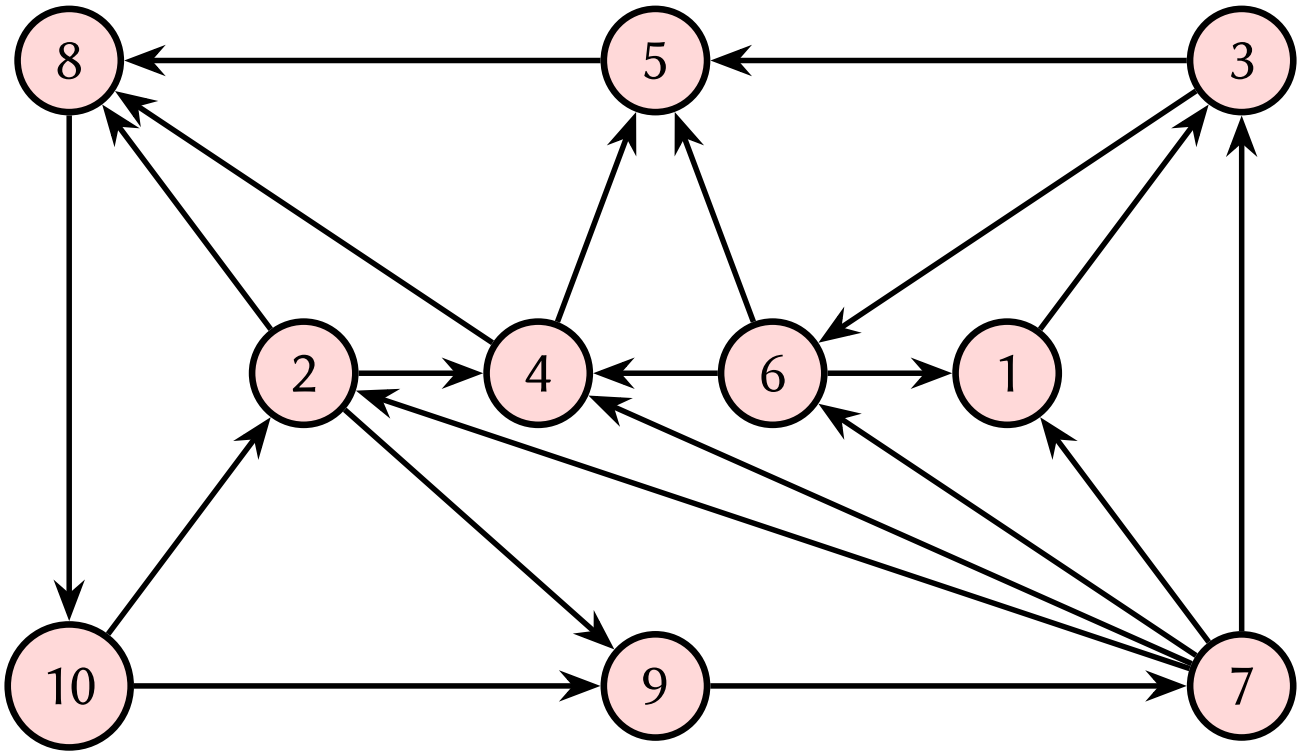
On obtient l'arborescence de parcours en profondeur



Numérotation préfixe (gris) DFS(0) : 0 5 3 1 2 8 4 6 7 | DFS(9) : 9 10
Numérotation suffixe (noir) DFS(0) : 8 2 1 4 7 6 3 5 0 | DFS(9) : 10 9

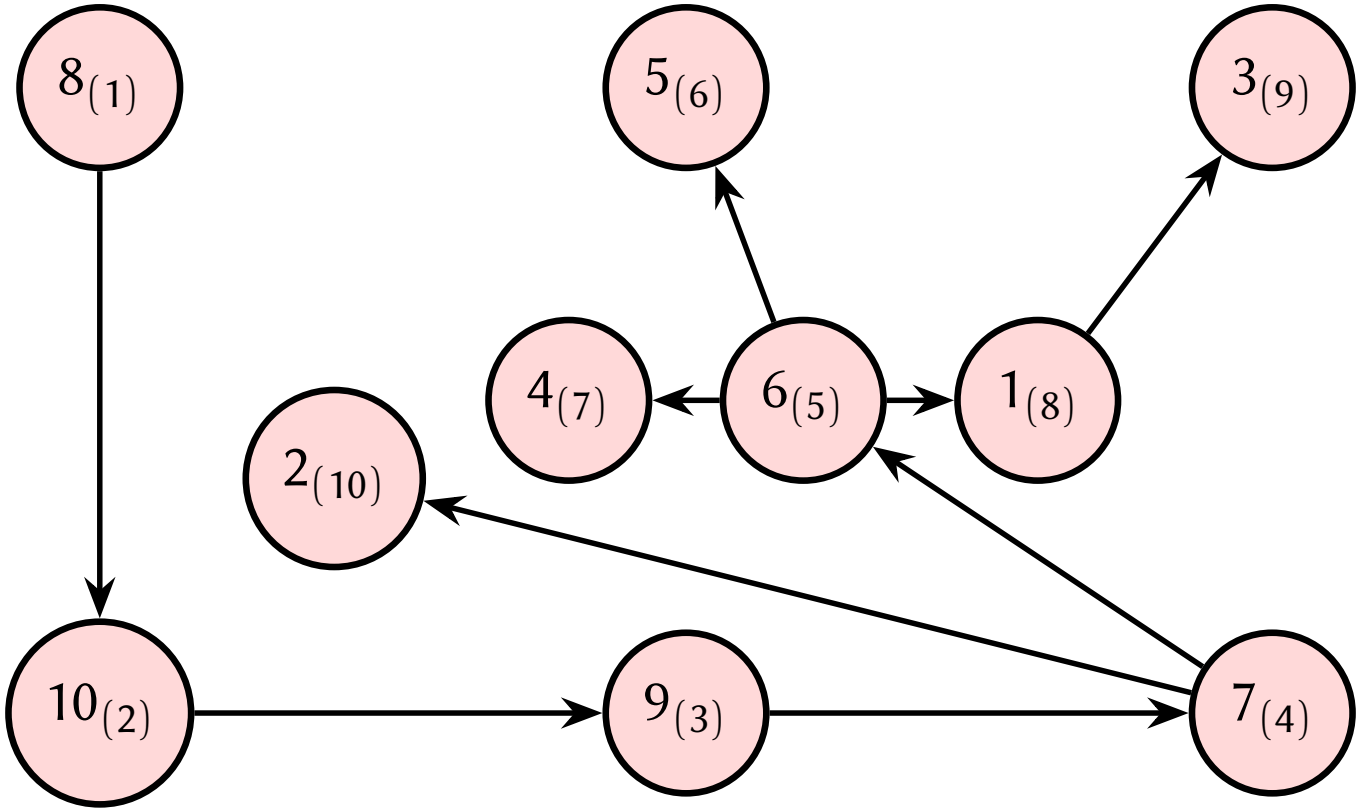
Parcours en profondeur d'abord (DFS)

Exercice : Réaliser le parcours en profondeur du graphe G, par ordre décroissant des sommets, depuis le sommet 8. Donner l'arbre de parcours et les numérotation préfixe et suffixe.



Parcours en profondeur d'abord (DFS)

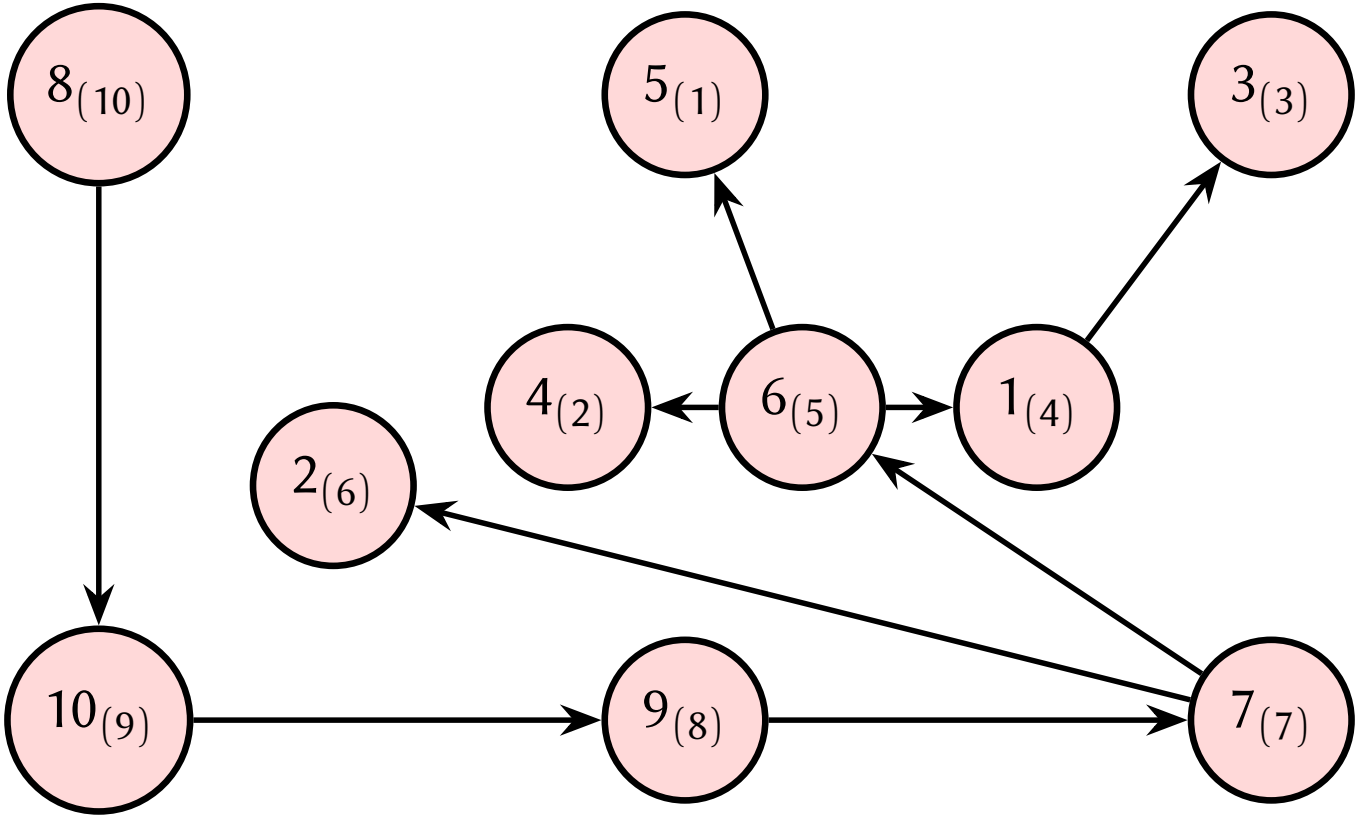
Donne l'arbre de parcours :



numérotation préfixe : 8 , 10 , 9 , 7 , 6 , 5 , 4 , 1 , 3 , 2

Parcours en profondeur d'abord (DFS)

Donne l'arbre de parcours :



numérotation suffixe : 5 , 4 , 3 , 1 , 6 , 2 , 7 , 9 , 10 , 8

Classification des arcs

Le parcours en profondeur sert à classer les arcs en différentes catégories. Cette classification des arcs peut servir à glaner d'importantes informations concernant le graphe. nous verrons qu'un graphe orienté est acyclique si et seulement si un parcours en profondeur n'engendre aucun arc « arrière ».

Classification des arcs

le parcours en profondeur d'un graphe G depuis le sommet u permet de définir quatre types d'arc :

- Les arcs de liaison : sont les arcs de la forêt de parcours en profondeur. C'estàdire, les arcs retenus pour le parcours en profondeur.
- Les arcs avants : sont les arcs (u, v) qui ne sont pas des arcs de liaison, et qui relient un sommet u à un descendant v dans une arborescence de parcours en profondeur.

Classification des arcs

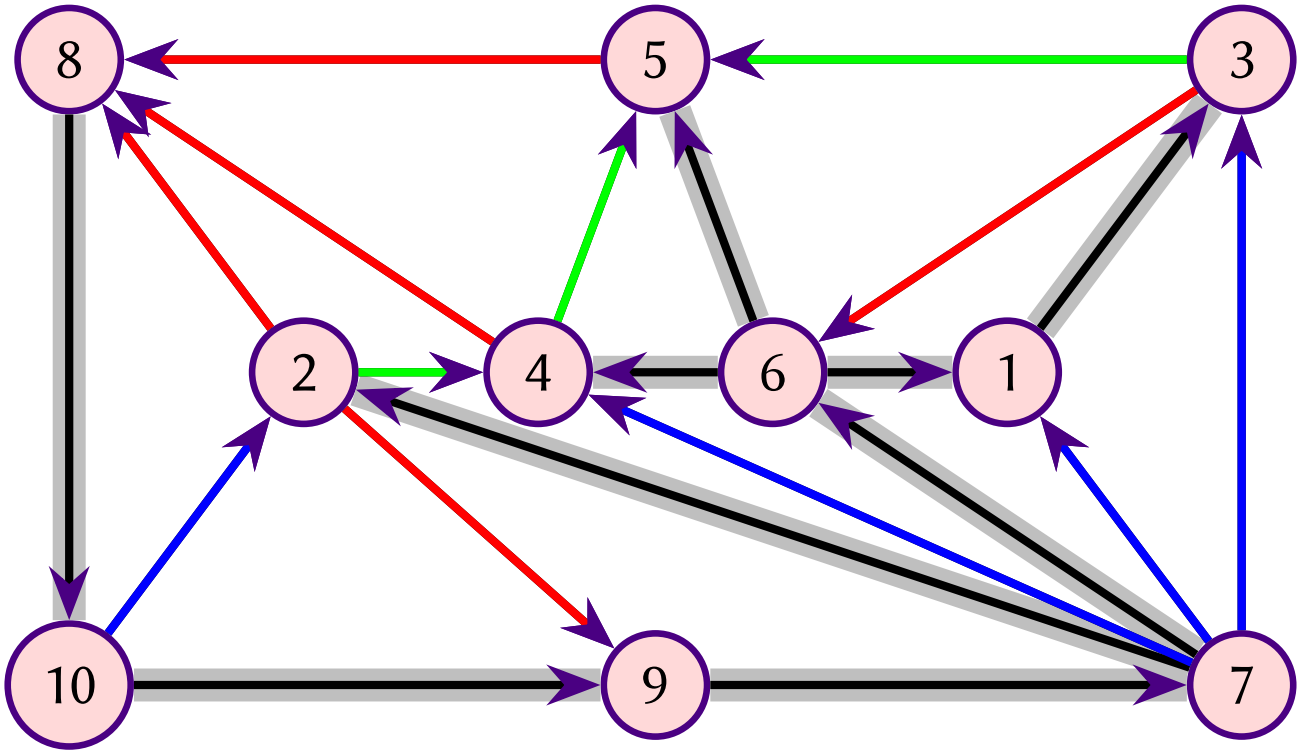
le parcours en profondeur d'un graphe G depuis le sommet u permet de définir quatre types d'arc :

- Les arcs arrières : sont les arcs (u, v) reliant un sommet u à un ancêtre v dans une arborescence de parcours en profondeur, ces arcs n'appartenant pas au parcours en profondeur. Les boucles (graphes orientés uniquement) sont considérées comme des arcs arrières.
- Les arcs transverses : sont tous les autres arcs. Ils peuvent relier deux sommets d'une même arbre de parcours en profondeur, du moment que l'un des sommets n'est pas un ancêtre de l'autre ; ils peuvent aussi relier deux sommets appartenant à des arborescences de parcours en profondeur différentes.

Classification des arcs

On réalisant le parcours en profondeur DFS du graphe G , par ordre décroissant des sommets, depuis le sommet 8.

arcs de liaison arcs avants arcs arrières arcs transverses



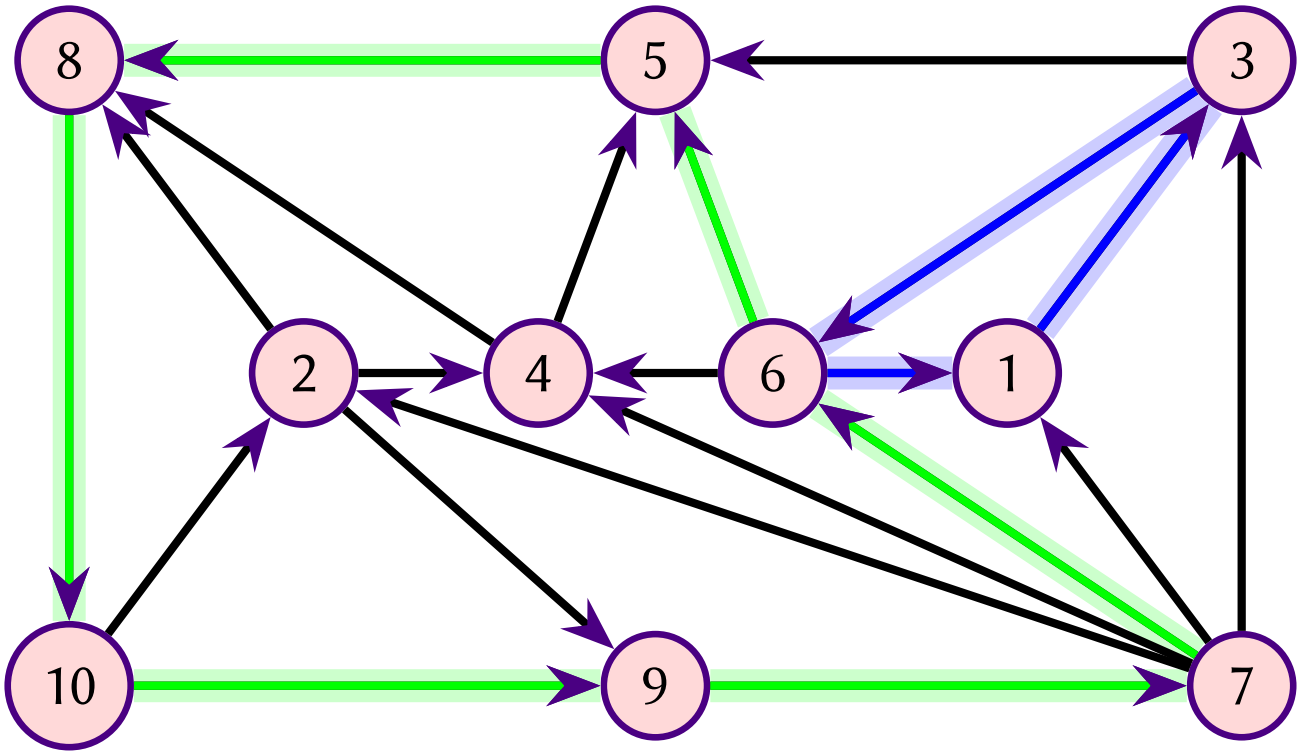
Cette classification permet de détecter des circuits dans un graphe.

Théorème(détection des circuits) : Dans un graphe $G = (S, A)$ et x un sommet. Si dans le parcours en profondeur de G à partir du sommet x il existe un arc arrière alors il existe au moins un circuit dans le graphe G.

Preuve : Soit (y, z) un arc arrière du graphe G (lors du parcours depuis x). Soit $y = z$ et alors cet arc est une boucle qui est le plus simple des circuits. Sinon il existe un chemin dans l'arbre de parcours qui va de x à y passant par z, donc on a un chemin $C^{'}$ de z à y qui permet de construire un circuit $C^{''}$. □

Classification des arcs

Exemple de détection de circuit du graphe G d'après le parcours en profondeur, par ordre décroissant des sommets, depuis le sommet 8 :



l'arc arrière (5, 8) permet de retrouver le circuit : (8, 10, 9, 7, 6, 5, 8).
l'arc arrière (3, 6) permet de retrouver le circuit : (3, 6, 1, 3).

Classification des arcs

Théorème. Dans un parcours en profondeur d'un graphe non orienté G, chaque arête de G est soit un arc de liaison, soit un arc arrière.

Classification des arcs

Algorithme 4 Classification des arcs d'un graphe orienté G

Require : Un graphe G orienté ;
Ensure : Classification des arcs du graphe G ;

```
Procedure DFS_Visit(u)
couleur[u] ← gris ;
d[u] ← time ← time + 1 ;
Pour Chaque Sommet v adjacent à u Faire
  Si couleur[v] == noir Alors
    Si d[u] < d[v] Alors
      Classer (u, v) comme arcs avant ;
    Sinon
      Classer (u, v) arcs transverses ;
      Si couleur[v] == Gris Alors
        Classify (u, v) arcs arrière ;
      fin Si
      Si couleur[v] == Blanc Alors
        π[v] ← u ;
        Classer (u, v) comme arcs de liaison ;
        DFS_Visit(v) ;
      fin Si
    fin Si
  fin Si
fin Pour
Couleur[u] ← Noir ;
f[u] ← time ← time + 1 ;
FinProcedure DFS_Visit
```

Classification des arcs

Algorithme 5 : DFS_DETECTION_CYCLES (G)

```
Procedure DFS_DETECTION_CYCLES(G)
Pour Chaque Sommet u ∈ S Faire
    couleur[u] ← Blanc;
    π[u] ← Nil;
fin Pour
t ← 0;
Pour Chaque Sommet u ∈ S Faire
    Si couleur[i] == Blanc Alors
        DFS_Visit(u);
    fin Si
fin Pour
FinProcedure DFS
Procedure DFS_Visit(u)
couleur[u] ← gris;
d[u] ← time ← time + 1;
Pour Chaque Sommet v adjacent à u Faire
    Si couleur[v] == Gris and π[u] ≠ v Alors
        Return "cycle exists"
    fin Si
    Si Couleur[v] == Blanc Alors
        π[v] ← u;
        DFS_Visit(v);
    fin Si
fin Pour
Couleur[u] ← Noir;
f[u] ← time ← time + 1;
FinProcedure DFS_Visit
```

Application DFS pour le tri topologique

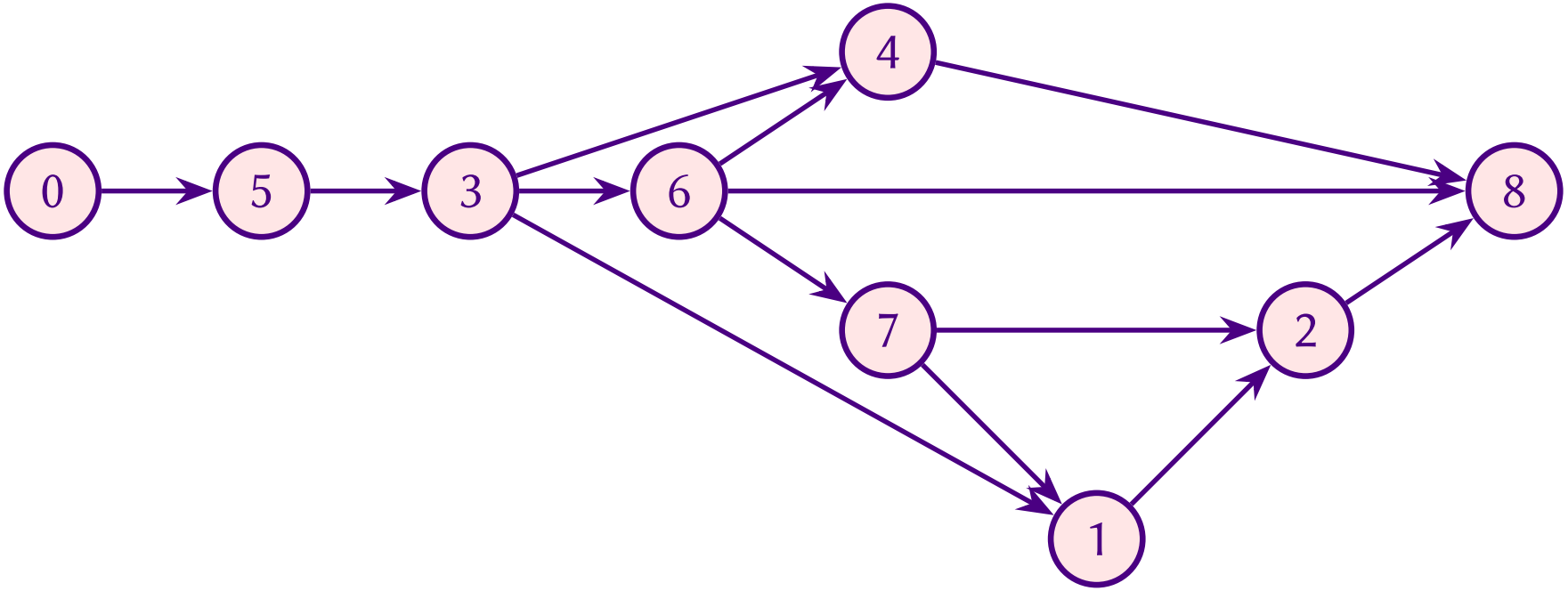
Algorithme 2 Tri topologique d'un graphe

1: **Procedure** Tri_Topologique(G)
2: Appeller **DFS**(G) *pour calculer les* f[u] ;
3: à chaque fois qu'un sommet est (noir), l'insérer dans une pile P ;
4: retouner P ;

Nous citons une application de la recherche en profondeur d'abord (DFS), mais il en existe d'autres : recherche de composantes connexes, de composantes fortement connexes, les points d'articulations, les bridges, . . .

On peut effectuer un tri topologique en $\theta(|S|+|A|)$, car le parcours en profondeur prend $\theta(|S|+|A|)$ et l'insertion de chacun des $|S|$ sommets dans la pile nécessite $\theta(1)$. L'algorithme reste de la même complexité, i.e. en $\theta(|S|+|A|)$.

Application DFS pour le tri topologique



List : *L*	0	5	3	6	7	4	1	2	8

Parcours en largeur d'abord (BFS : Breadth First Search)

Principe du parcours en largeur d'un graphe :

Étant donnés un graphe $G = (S, A)$ et un sommet origine s, le parcours en largeur emprunte systématiquement les arcs de G pour « découvrir » tous les sommets accessibles depuis s. Il calcule la distance (plus petit nombre d'arcs) entre s et tous les sommets accessibles. Il construit également une « arborescence de parcours en largeur » de racine s, qui contient tous les sommets accessibles depuis s.

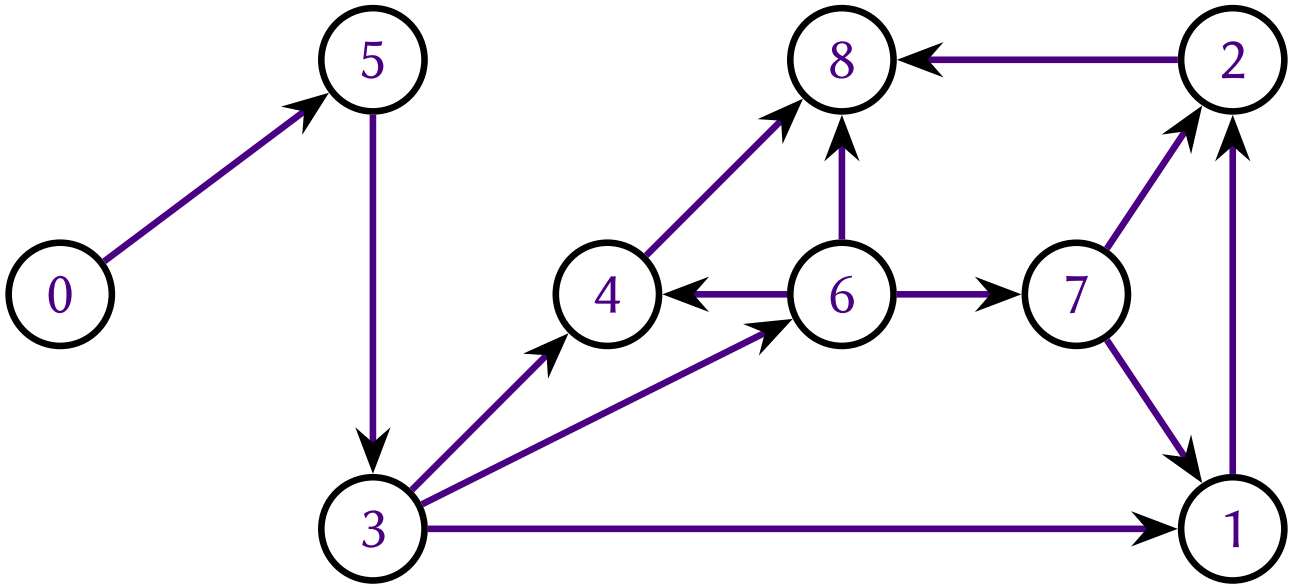
On part d'un sommet donné. On énumère tous les fils (les suivants) de ce sommet, puis tous les petits-fils non encore énumérés, etc. C'est une énumération par génération : les successeurs directs, puis les successeurs au 2e degré, etc.
L'algorithme est globalement en $O(|S| + |A|)$. Donc linéaire dans la taille du graphe (sous forme de listes d'adjacence).

Parcours en largeur d'abord (BFS)

Algorithme 3 Parcours en largeur d'un graphe

```
1:  Procedure BFS(G,s)
2:     Pour chaque sommet u ∈ S − {s} faire {
3:        couleur[u] ← b ;
4:        d[u] = +∞
5:        π[u] ← NIL ; }
6:     couleur[s] ← g ;   d[s] ← 0 ;   π[s] ← NIL ; Enfiler(Q,s) ;
7:     Tant que Q ≠ ∅ faire {
8:        u = tete(Q) ;
9:        Pour chaque sommet v ∈ Adj[u] faire
10:          si couleur[v] = b alors {
11:             π[v] ← u ;
12:             couleur[v] ← g ;
13:             d[v] ← d[u] + 1 ;
14:             Enfiler(Q, v) ; }
15:       Défiler(Q) ;
16:       couleur[u] ← n ; }
```

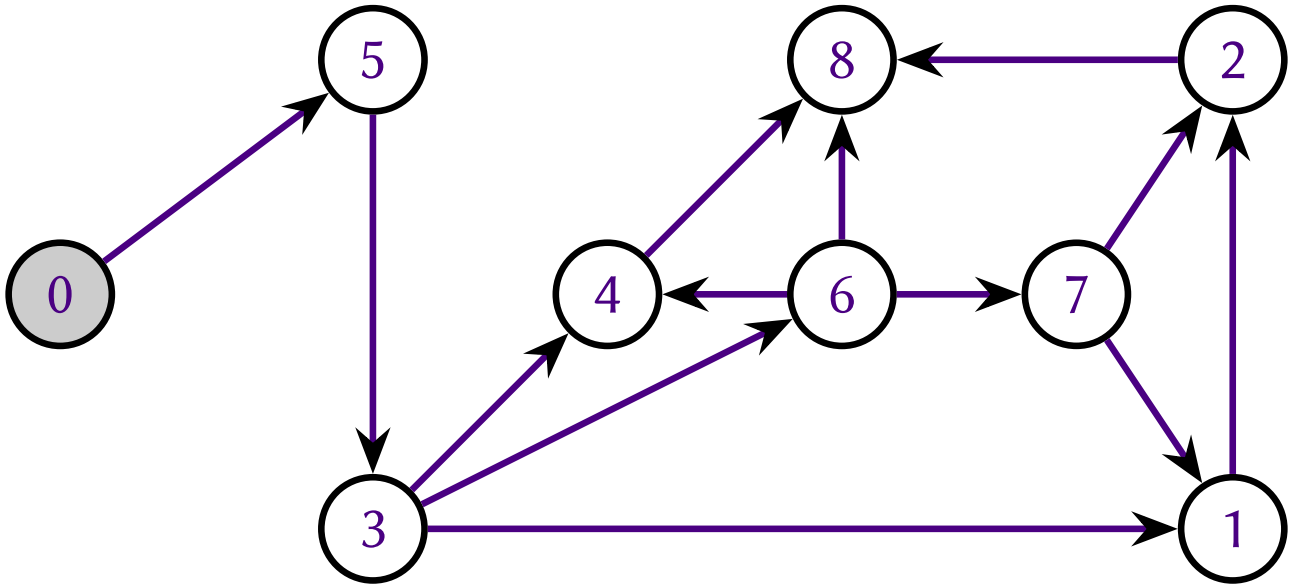
Parcours en largeur d'abord (BFS)



	0	1	2	3	4	5	6	7	8
Couleur	B	B	B	B	B	B	B	B	B
d	∞	∞	∞	∞	∞	∞	∞	∞	∞
File : Q									

Output :

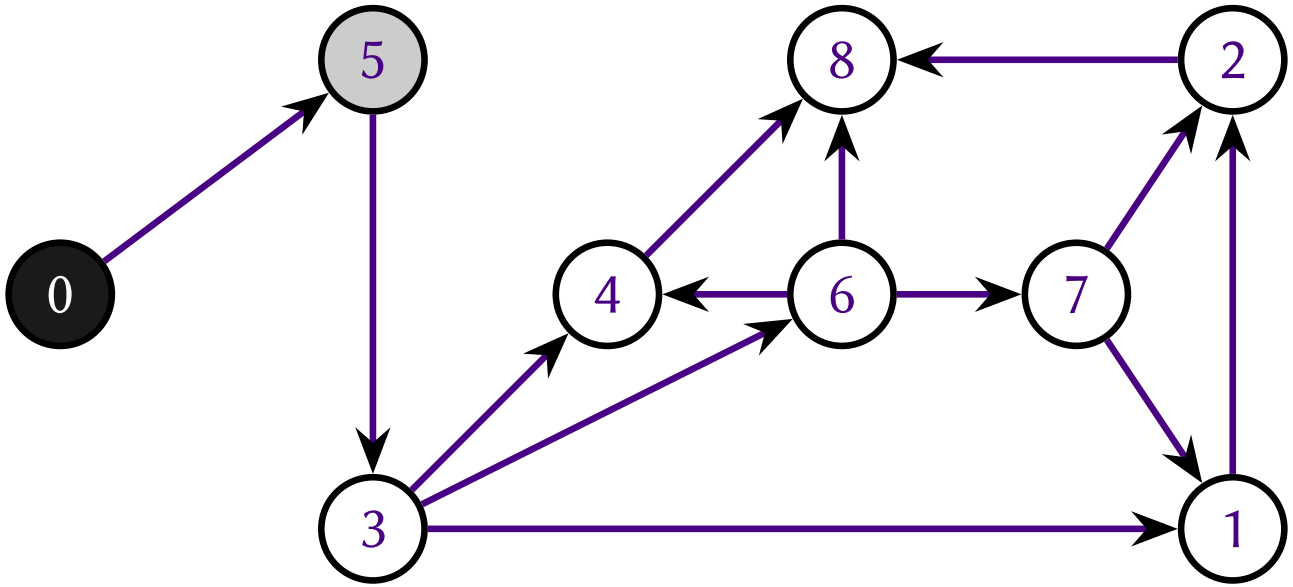
Parcours en largeur d'abord (BFS)



	0	1	2	3	4	5	6	7	8
Couleur	G	B	B	B	B	B	B	B	B
d	0	∞	∞	∞	∞	∞	∞	∞	∞
File : Q	0								

Output : 0

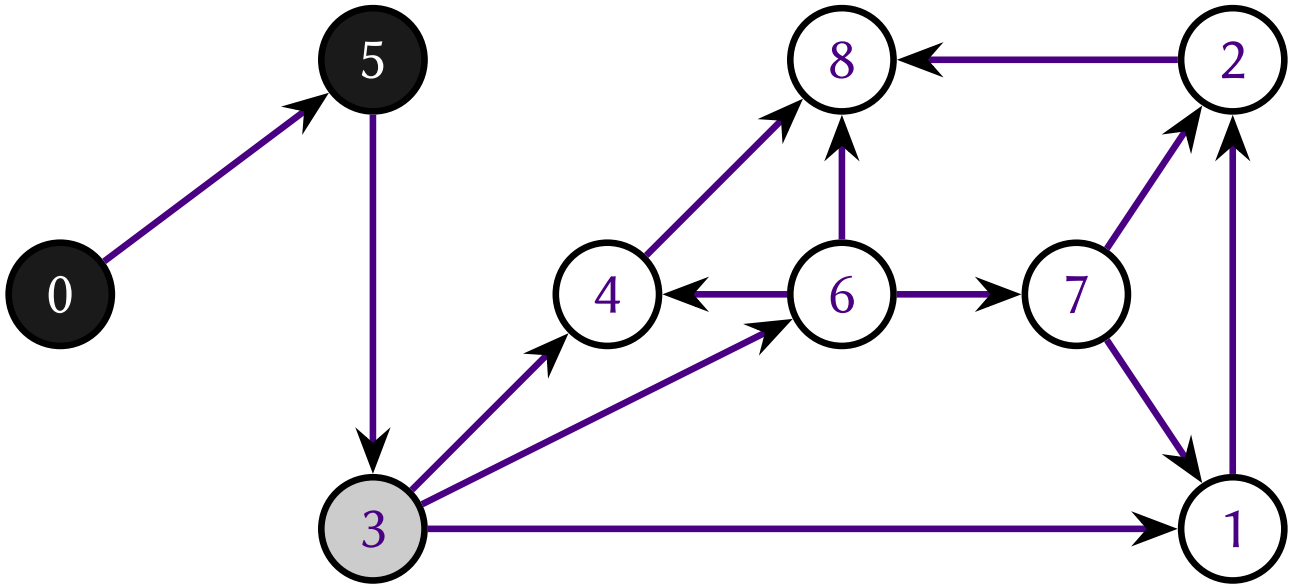
Parcours en largeur d'abord (BFS)



	0	1	2	3	4	5	6	7	8
Couleur	N	B	B	B	B	G	B	B	B
d	0	∞	∞	∞	∞	1	∞	∞	∞
File : Q		5							

Output : 0 5

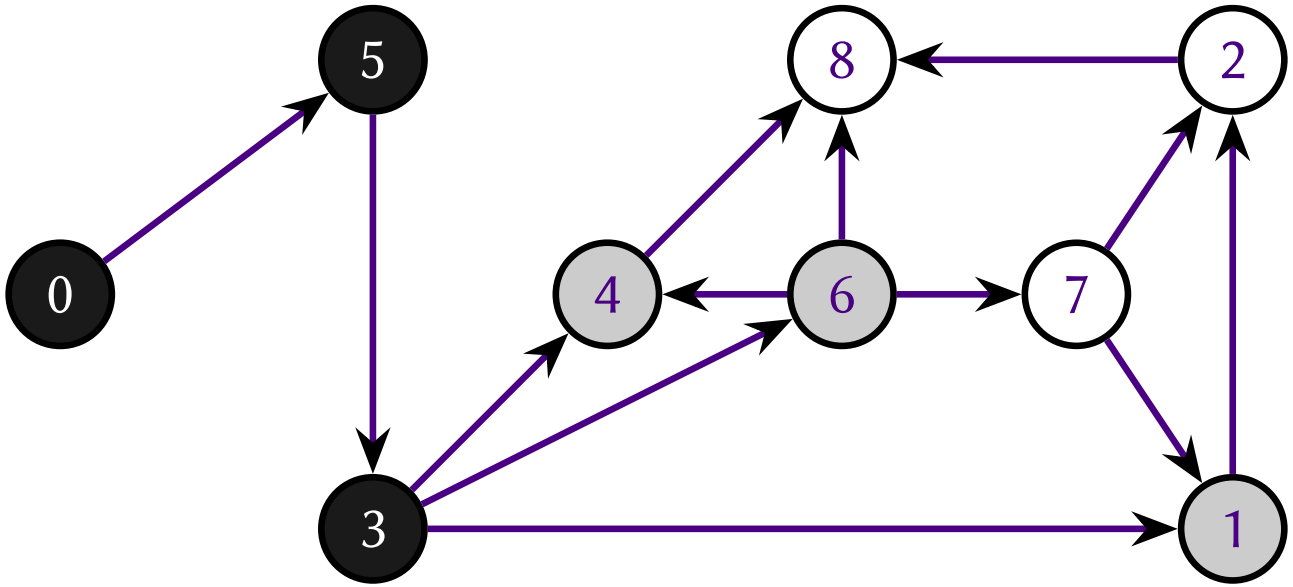
Parcours en largeur d'abord (BFS)



	0	1	2	3	4	5	6	7	8
Couleur	N	B	B	G	B	N	B	B	B
d	0	∞	∞	2	∞	1	∞	∞	∞
File : Q		3							

Output : 0 5 3

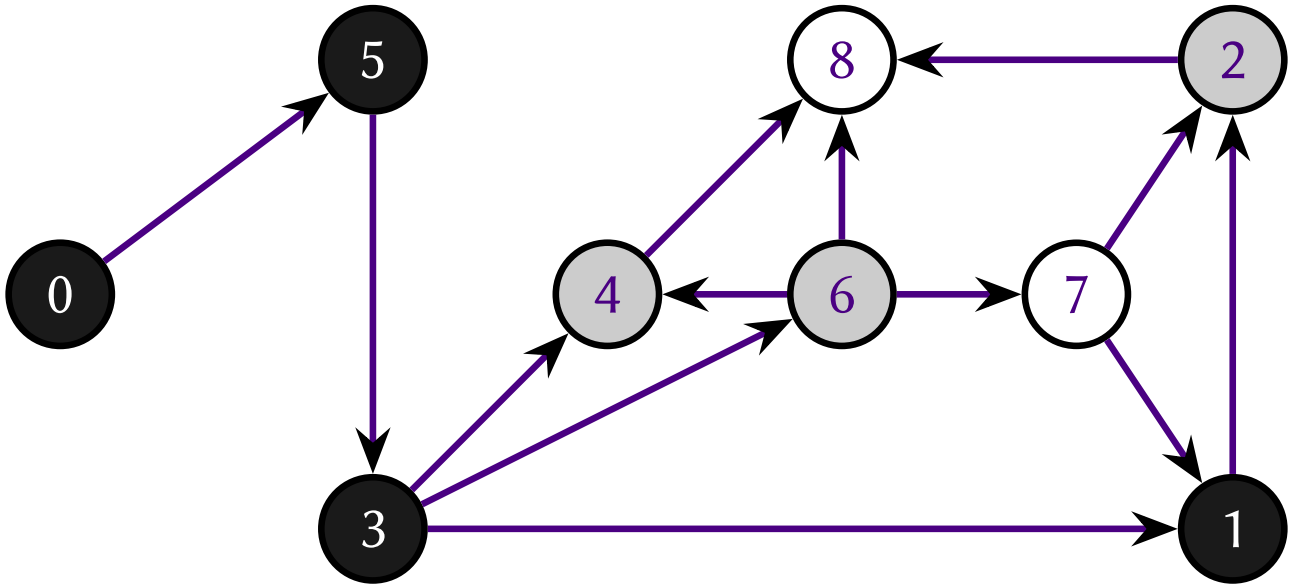
Parcours en largeur d'abord (BFS)



	0	1	2	3	4	5	6	7	8
Couleur	N	G	B	N	G	N	G	B	B
d	0	3	∞	2	3	1	3	∞	∞
File : Q		1	4	6					

Output : 0 5 3 1 4 6

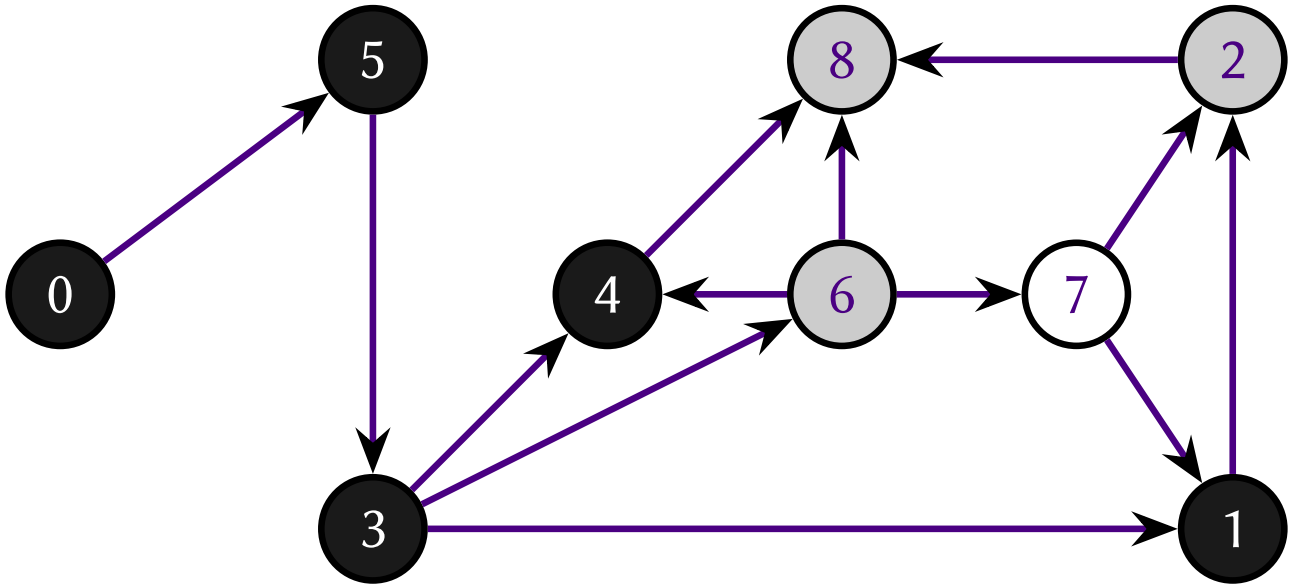
Parcours en largeur d'abord (BFS)



	0	1	2	3	4	5	6	7	8
Couleur	N	N	G	N	G	N	G	B	B
d	0	3	4	2	3	1	3	∞	∞
File : Q			4	6	2				

Output : 0 5 3 1 4 6 2

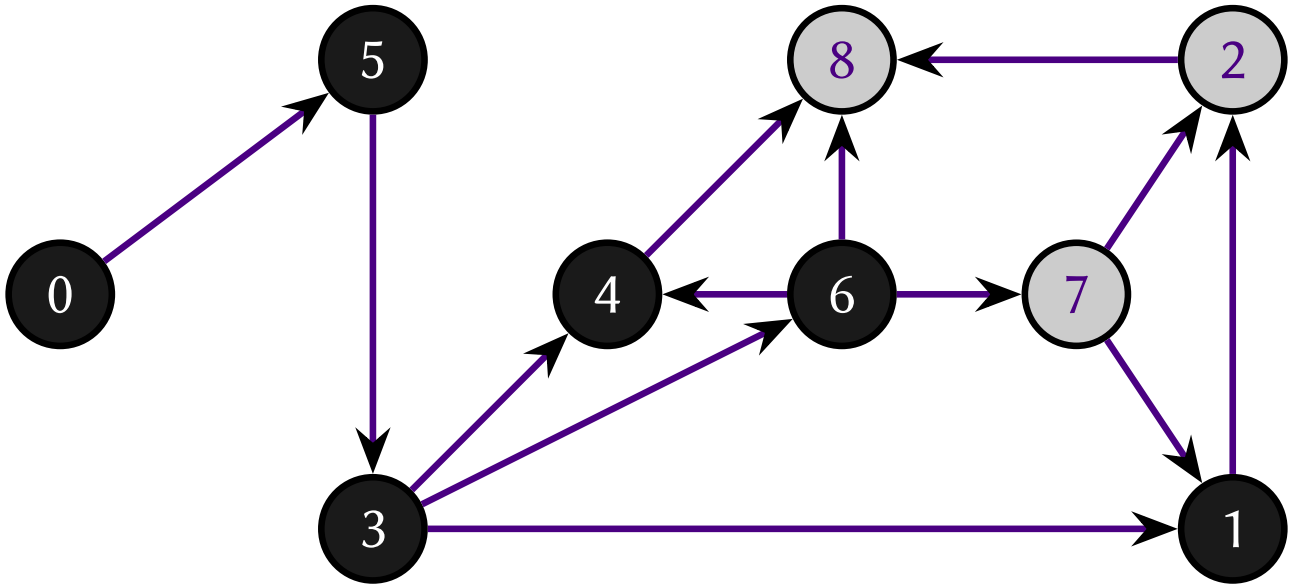
Parcours en largeur d'abord (BFS)



	0	1	2	3	4	5	6	7	8
Couleur	N	N	G	N	N	N	G	B	G
d	0	3	4	2	3	1	3	∞	4
File : Q				6	2	8			

Output : 0 5 3 1 4 6 2 8

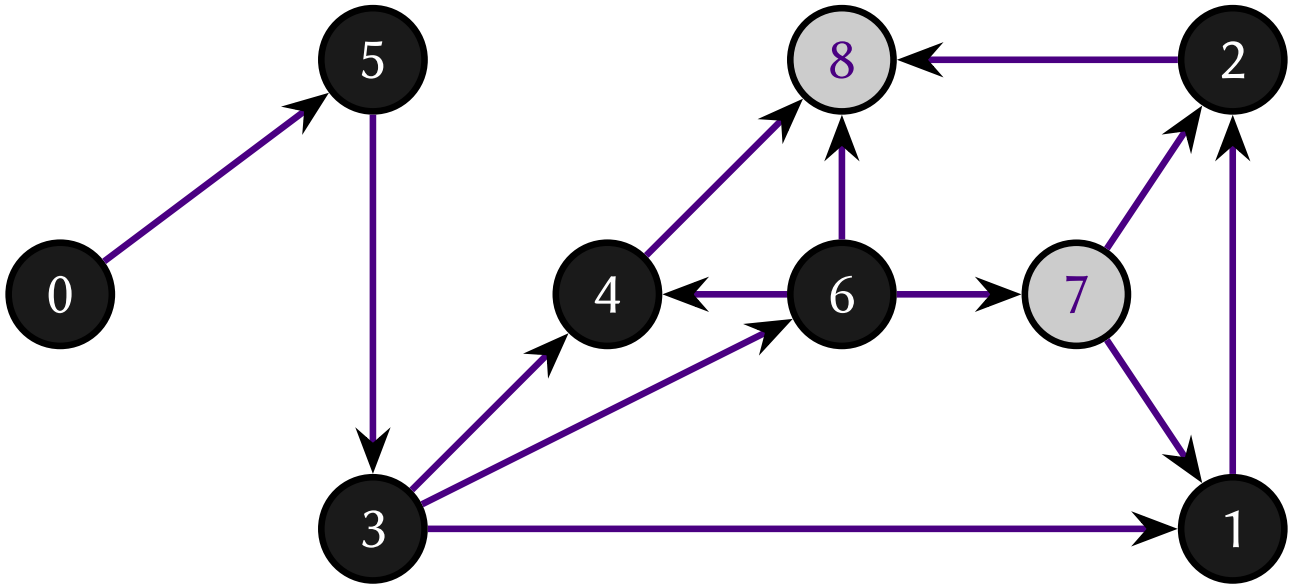
Parcours en largeur d'abord (BFS)



	0	1	2	3	4	5	6	7	8
Couleur	N	N	G	N	N	N	N	G	G
d	0	3	4	2	3	1	3	4	4
File : Q					2	8	7		

Output : 0 5 3 1 4 6 2 8 7

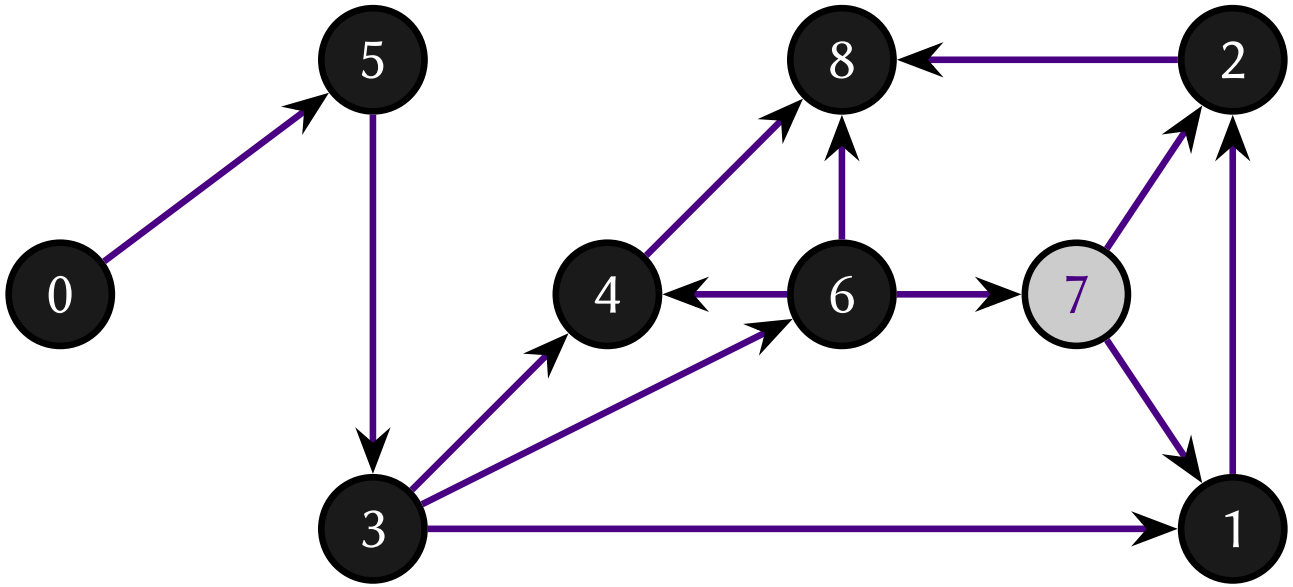
Parcours en largeur d'abord (BFS)



	0	1	2	3	4	5	6	7	8
Couleur	N	N	N	N	N	N	N	G	G
d	0	3	4	2	3	1	3	4	4
File : Q						8	7		

Output : 0 5 3 1 4 6 2 8 7

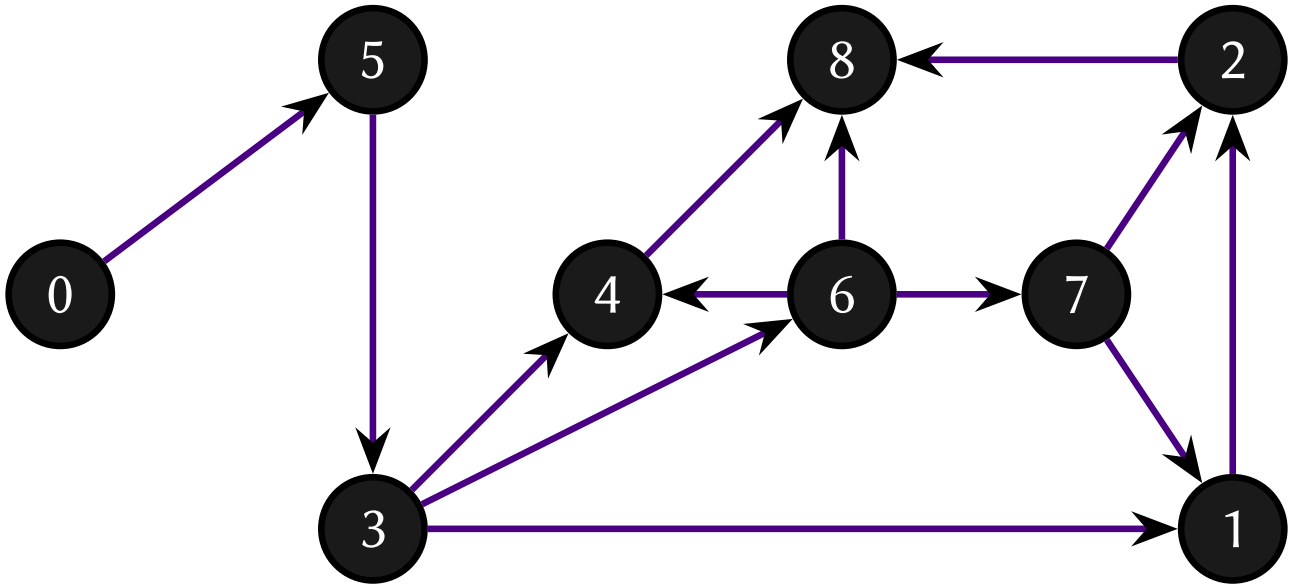
Parcours en largeur d'abord (BFS)



	0	1	2	3	4	5	6	7	8
Couleur	N	N	N	N	N	N	N	G	N
d	0	3	4	2	3	1	3	4	4
File : Q							7		

Output : 0 5 3 1 4 6 2 8 7

Parcours en largeur d'abord (BFS)

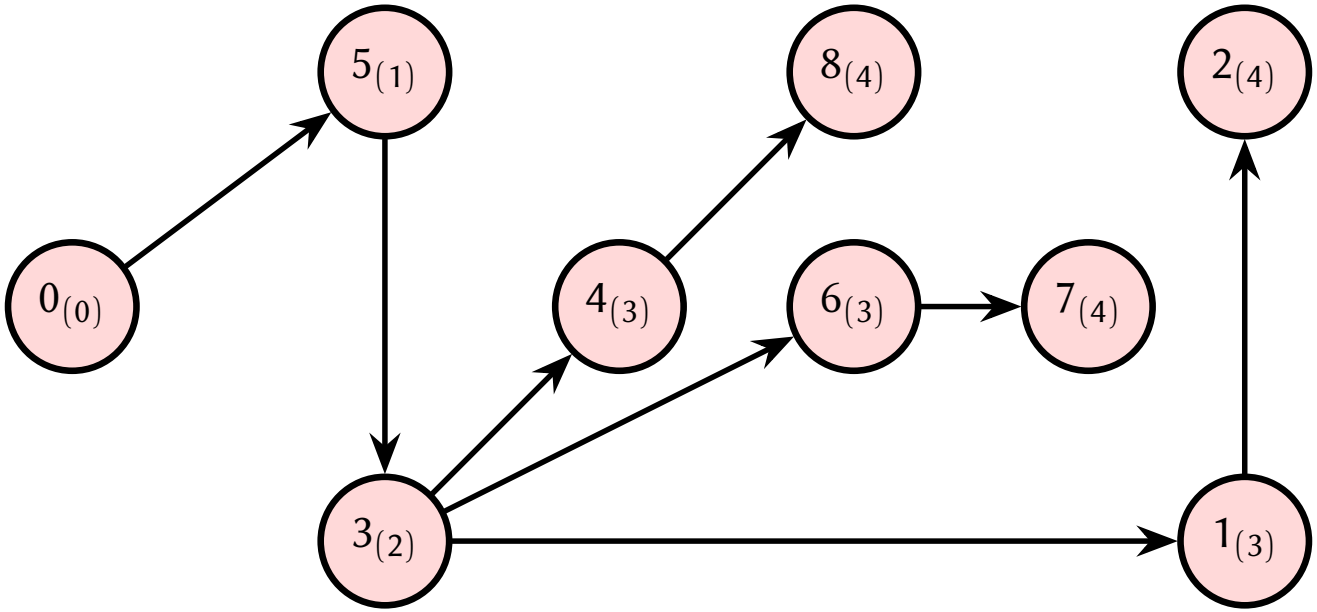


	0	1	2	3	4	5	6	7	8
Couleur	N	N	N	N	N	N	N	N	N
d	0	3	4	2	3	1	3	4	4
File : Q									

Output : 0 5 3 1 4 6 2 8 7

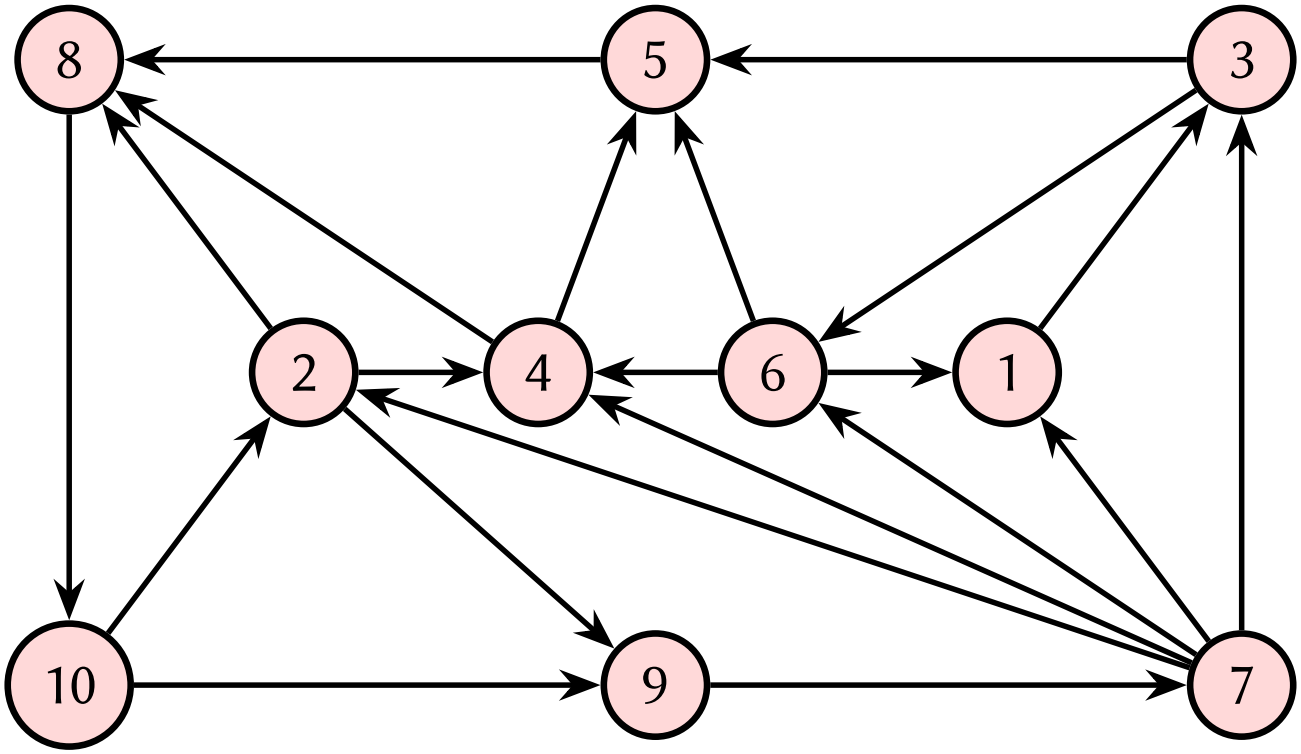
Parcours en largeur d'abord (BFS)

L'arborescence de parcours en largeur :



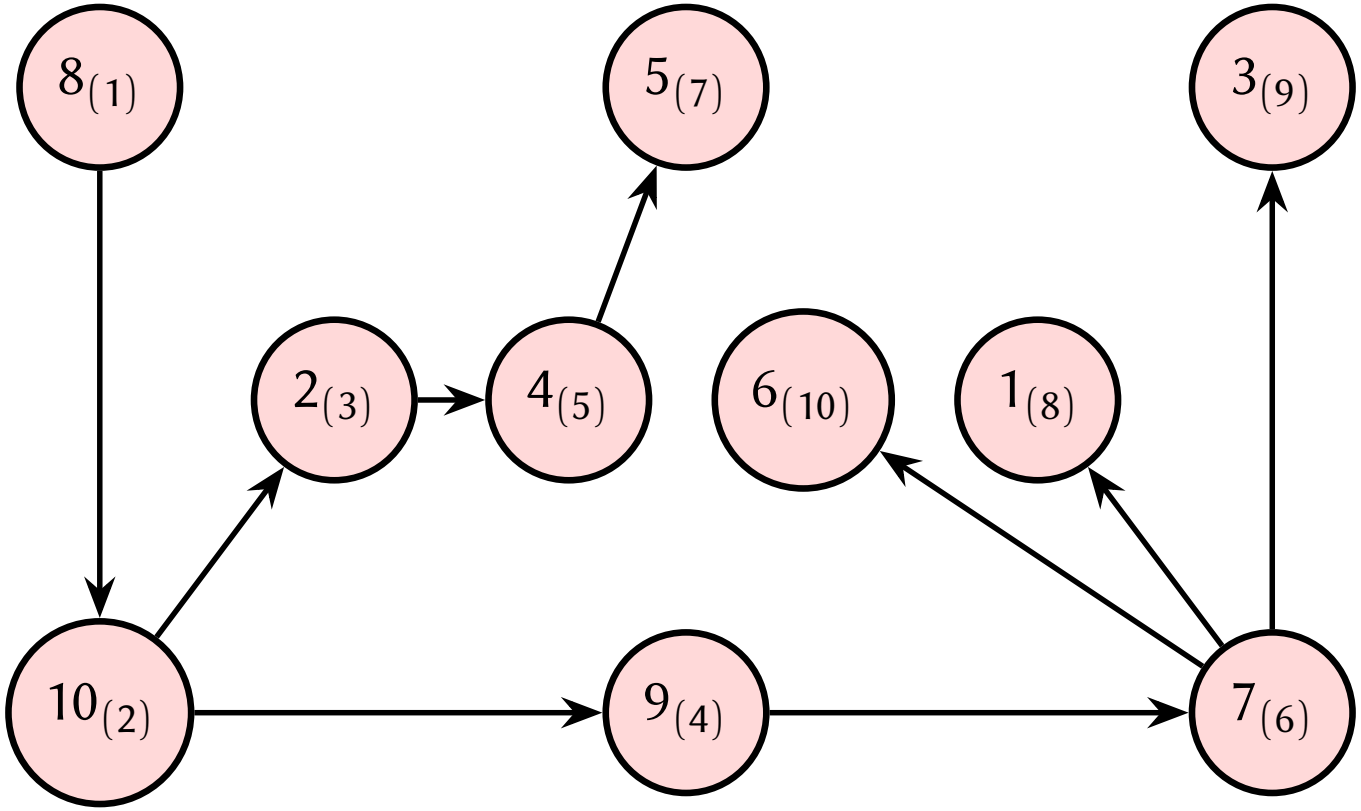
Parcours en largeur d'abord (BFS)

Exercice : Réaliser le parcours en largeur BFS du graphe G , par ordre croissant des sommets, depuis le sommet 8.



Parcours en largeur d'abord (BFS)

L'arborescence de parcours en largeur :



Parcours en largeur d'abord (BFS)

```
#include <iostream>
#include <iomanip>
#include <vector>
#include <list>
#include <queue>

#define infini 999999
#define Nil -1
#define Blanc 'B'
#define Gris 'G'
#define Noir 'N'

using namespace std;

vector<char> couleur;
vector<int> d;
vector<int> Pi;

// ******************************** BFS ****************************************

void BFS(vector<list<int> > G, int s)
{
    int u, v;
    list<int>::iterator index;
    queue<int> Q;
    couleur[s] = Gris;
    d[s] = 0;
    Pi[s] = Nil;
    Q.push(s);
```

Parcours en largeur d'abord (BFS)

```
    while (!Q.empty()) {
        u = Q.front();
        for (index = G[u].begin(); index != G[u].end(); ++index) {
            v = *index;
            if (couleur[v] == Blanc) {
                couleur[v] = Gris; // On vient de découvrire le sommet v
                d[v] = d[u] + 1;
                Pi[v] = u;
                Q.push(v);
            }
        }
        Q.pop();
        couleur[u] = Noir; // On a terminer avec le sommet u
        cout << "  " << u << " "; // traitement du sommet u
    }
    cout << endl;
}
// ****************************************************************************

void initialisation_graphe(vector<list<int> >& Graphe)
{
    Graphe[0].push_back(5);
    Graphe[5].push_back(3);
    Graphe[3].push_back(1);
    Graphe[3].push_back(4);
    Graphe[3].push_back(6);
    Graphe[1].push_back(2);
    Graphe[4].push_back(8);
    Graphe[6].push_back(4);
```

Parcours en largeur d'abord (BFS)

```
    Graphe[6].push_back(7);
    Graphe[6].push_back(8);
    Graphe[7].push_back(1);
    Graphe[7].push_back(2);
    Graphe[2].push_back(8);
    Graphe[9].push_back(10);
}

// ***************************************************************************

int main()
{
    unsigned N = 11; // le nombre de Sommets du graphe
    vector<list<int> > Graphe(N); // déclaration du graphe en list d'adjacences
    initialisation_graphe(Graphe);

// ***************************************************************************

    for (unsigned i = 0; i < Graphe.size(); i++) {
        couleur.push_back(Blanc);
        d.push_back(infini);
        Pi.push_back(Nil);
    }

// ***************************************************************************

    int compteur1 = 0;
    cout << "\n\n  Le Graphe représenter par les listes d'adjacences :\n\n";
    for (unsigned i = 0; i < Graphe.size(); ++i) {
        list<int>::iterator iter;
```

Parcours en largeur d'abord (BFS)

```
        cout << "\tGraphe [" << i << "] \t: ";
        for (iter = Graphe[i].begin(); iter != Graphe[i].end(); ++iter) {
            cout << "(" << *iter << ")"
                 << "->";
            compteur1++;
        }
        cout << "(/)" << endl;
    }
    cout << "\n  Nombre d'arc : " << compteur1 << "\n";
    cout << "  Nombre de sommets : " << Graphe.size() << "\n\n";

// ***************************************************************************

    cout << "  Parcours en Largeur du graphe : \n\n";

    for (unsigned i = 0; i < Graphe.size(); i++)

    if(couleur[i]==Blanc)  {
        cout << "                ";
        BFS(Graphe, i);
    }

// ***************************************************************************
    cout << "\n  Les Sommets     :\t"
         << "| ";
    for (unsigned j = 0; j < Graphe.size(); j++)
        cout << setw(2) << j << " | ";
    cout << endl;
    cout << "  Arborescence    :\t"
         << "| ";
```

Parcours en largeur d'abord (BFS)

```
    for (unsigned j = 0; j < Graphe.size(); j++)
        cout << setw(2) << Pi[j] << " | ";
    cout << endl;
    cout << "  Les distances   :\t"
         << "| ";
    for (unsigned i = 0; i < Graphe.size(); i++) {
        if (d[i] == infini)
            cout << setw(2) << "X"
                 << " | ";
        else
            cout << setw(2) << d[i] << " | ";
    }
    cout << endl;

    cout << "  Couleur         :\t"
         << "| ";
    for (unsigned j = 0; j < Graphe.size(); j++)
        cout << setw(2) << couleur[j] << " | ";
    cout << endl;

    return 0;
}
```

Parcours en largeur d'abord (BFS)

Le Graphe représenter par les listes d'adjacences :

```
Graphe [0]   : (5)->(/)
Graphe [1]   : (2)->(/)
Graphe [2]   : (8)->(/)
Graphe [3]   : (1)->(4)->(6)->(/)
Graphe [4]   : (8)->(/)
Graphe [5]   : (3)->(/)
Graphe [6]   : (4)->(7)->(8)->(/)
Graphe [7]   : (1)->(2)->(/)
Graphe [8]   : (/)
Graphe [9]   : (10)->(/)
Graphe [10]  : (/)

Nombre d'arc : 14
Nombre de sommets : 11

Parcours en Largeur du graphe :

             0   5   3   1   4   6   2   8   7
             9   10

Les Sommets     :  |  0 |  1 |  2 |  3 |  4 |  5 |  6 |  7 |  8 |  9 | 10 |
Arborescence    :  | -1 |  3 |  1 |  5 |  3 |  0 |  3 |  6 |  4 | -1 |  9 |
Les distances   :  |  0 |  3 |  4 |  2 |  3 |  1 |  3 |  4 |  4 |  0 |  1 |
Couleur         :  |  N |  N |  N |  N |  N |  N |  N |  N |  N |  N |  N |
```

Merci de votre attention